

RoboDog

Model-Derived Design, ROS 2 Architecture, Locomotion Control,
and Simulation Evaluation of a 28 kg Quadruped Robot

12 × ROBSTRIDE06 actuators • 2:1 knee transmissions • RGB-D mapping

Generated from the Onshape CAD export `b95d92fd9159f4f7`

Revision 1.0 — scientific system report; 7 October 2026

Abstract

This work presents the design, implementation and simulation evaluation of RoboDog, a 28.000 kg, 12-degree-of-freedom quadruped built around twelve ROBSTRIDE06 actuators. A reproducible model pipeline transforms a CAD-derived URDF into consistent kinematic, inertial, ROS 2 and MuJoCo descriptions. Each knee uses a 2:1 external belt transmission with 95 % assumed efficiency; the hips remain direct drive. The controller combines phase-based gait scheduling, continuous stance and swing trajectories, Raibert-style foot placement, stance-force distribution, inverse kinematics, joint impedance and an unconditional safety layer at 400 Hz. Physics runs at 2 kHz and applied actuator torque is sampled every physics step. Registered RGB-D data feed RTAB-Map graph SLAM and graph-corrected OctoMap products for persistent room scans. In 100 s, 1.0 m/s flat-ground tests, walk failed stability; heading-held trot remained upright and achieved 0.923 m/s, with a worst per-motor RMS torque of 6.749 N·m and a 34.897 N·m peak. Safety limiting occurred during 87.965 % of trot control cycles, so the operating point failed the tracking and motor-margin criteria. The results establish a reproducible simulation and mapping architecture, but do not establish hardware feasibility, validated thermal margin or reliable 1.0 m/s motion at 28 kg.

Keywords: quadruped robot; legged locomotion; ROS 2; impedance control; gait generation; MuJoCo; RGB-D SLAM; RTAB-Map; OctoMap; actuator sizing.

Contents

1	Introduction	3
1.1	Notation and abbreviations	3
2	Background and related work	3
2.1	Local evidence map	5
3	Robot platform and design requirements	5
3.1	Design targets	6
3.2	Mass budget	6
3.3	Coordinate systems and conventions	6
4	Model generation and robot mathematics	7
4.1	CAD to URDF pipeline	7
4.2	Kinematics	7
4.3	Knee drive topology	8
4.4	Joint limits	8
5	Actuation and electrical system	8
5.1	Specification and derived quantities	8
5.2	CAN interface	9
5.3	Bus budget	9

6	Locomotion and trajectory generation	10
6.1	The loop	10
6.2	Impedance control	11
6.3	Gravity feed-forward and the force convention	11
6.4	Gait generation	11
6.4.1	What the generator actually does	12
6.4.2	The phase model	12
6.4.3	Stance: the foot is a fixed point in the world	13
6.4.4	Swing: interpolate between two latched endpoints	13
6.4.5	Foot placement: where velocity is really decided	13
6.4.6	Three things regulate speed, and they are not interchangeable	14
6.4.7	Force distribution	14
6.4.8	Per-cycle summary	14
6.5	Balance	14
6.6	Historical RS02 locomotion performance	15
6.6.1	How it was broken, and what that cost	15
6.6.2	What the balance layer is for, now	16
6.7	Historical RS02 thermal load	16
6.8	State estimation	17
7	Safety and load protection	17
7.1	Two-tier limits	17
7.2	Design rules	18
7.3	Torque limiting under impedance control	18
7.4	Thermal model	18
7.5	Watchdog and emergency stop	19
8	ROS 2 software and data architecture	20
8.1	Packages	20
8.2	Package-level architecture in depth	20
8.3	The hardware boundary	21
8.4	Interfaces	22
9	Simulation model and test environments	23
9.1	Fidelity choices	23
9.2	Validation against the model	24
9.3	Test environment	24
10	Perception, SLAM and three-dimensional mapping	25
10.1	RGB-D SLAM and 3D occupancy	26
11	Operator interfaces and telemetry	27
11.1	Joint load and history conventions	28
11.2	LAN discovery and mobile application	29
12	Experimental methodology and results	30
12.1	Contributions and research basis	31
12.2	Current RS06 simulation measurements	32
13	Hardware migration and commissioning	34
13.1	What changes	34
13.2	Bring-up sequence	34
13.3	Known gaps	35
13.4	Planned next steps	35

14 Reproducibility and verification	35
14.1 Simulation quick start	36
14.2 Analysis tools	36
14.3 Capability envelope	36
15 References	37

1 Introduction

Quadruped robots exchange the static simplicity of wheels for discontinuous contact, a floating body and a larger reachable set of footholds. Their controller must decide which feet support the body, where the other feet land, which ground-reaction forces are admissible, and which joint torques realize those forces without exceeding actuator or thermal limits. These problems are coupled to state estimation, rigid-body modeling and the timing behavior of the software graph.

This report asks four system-level questions. First, can one source model keep the CAD-derived mass properties, URDF, controller kinematics and MuJoCo plant consistent? Second, can the present controller move the 28 kg configuration at 1.0 m/s within the ROBSTRIDE06 envelope? Third, can shared ROS 2 contracts preserve command and telemetry meaning across ideal, MuJoCo and CAN backends? Fourth, can the simulated RGB-D pipeline build and persist coherent 2D and 3D maps? The contributions are a traceable model-generation pipeline, a backend-independent control and safety architecture, a physics-rate actuator load method, and an integrated SLAM/occupancy/export path.

The evidence boundary is central to every conclusion. No actuator, battery, IMU or camera has been commissioned on the physical robot. The 28 kg test adds a centered point-mass ballast to a 19.719271 kg base model, not a measured payload inertia. MuJoCo provides ground-truth odometry. Temperature is an uncalibrated observer output. Accordingly, this is a reproducible simulation study and software architecture report, not hardware qualification.

1.1 Notation and abbreviations

Symbol	Unit	Meaning
$q, \dot{q}, \tau$	rad, rad/s, N·m	joint position, velocity and output torque
$\mathbf{p}, \dot{\mathbf{p}}$	m, m/s	foot position and velocity in the stated frame
$\mathbf{J}$	—	analytic geometric Jacobian, $\partial \mathbf{p} / \partial \mathbf{q}$
$\mathbf{f}$	N	force exerted by the environment on a foot
r, η	—, —	motor-output/joint speed ratio and belt efficiency
D, f_g, ϕ	—, Hz, —	duty factor, gait frequency and normalized phase
K_t, R_ϕ	N·m/A, Ω	output torque constant and equivalent phase resistance
HAA/HFE/KFE	—	hip ab/adduction, hip flexion/extension, knee flexion/extension
RMS	—	root mean square over the explicitly stated time window
SLAM/TF/QoS	—	simultaneous localization and mapping; transform tree; quality of service

2 Background and related work

RoboDog is a torque-controlled, floating-base quadruped: four identical three-DOF legs support and propel a rigid body through intermittent ground contact. The gait controller is deliberately classical rather than learned: a phase scheduler chooses stance and swing legs; a foot-trajectory generator places feet; a body stabiliser requests ground-reaction forces; the analytic Jacobian maps those foot forces to joint torques; and the RS06 actuator closes its local impedance loop.

This split follows Raibert’s separation of legged balance into speed, attitude and height regulation, while keeping the implementation small enough to verify line by line [1, 2]. Modern platforms such as MIT Cheetah 3 combine high-bandwidth proprioceptive actuation with more sophisticated model-based control. RoboDog does not implement model-predictive or whole-body optimal control; those systems are used here as context rather than as a description of this controller [18, 19].

The mathematical core uses established robotics tools. Foot forces are converted to actuator torques with the Jacobian-transpose relationship used by operational-space control [3], and joint commands are expressed as impedance setpoints: target position, target velocity, stiffness, damping and feed-forward torque [4]. The controller therefore does not command raw current or a single position target; it commands a local dynamic relationship and then measures whether the simulated actuator effort, safety limiting and thermal estimates remain credible.

The ROS 2 architecture is a typed graph of nodes, topics and services. ROS 2 documentation defines nodes as graph participants that publish/subscribe topics, serve/request services and expose parameters;¹ topics are typed publish/subscribe streams for continuous data [5, 6]; and QoS policies such as reliability and durability determine whether a publisher/subscriber pair can exchange messages [7]. RoboDog keeps high-rate joint control inside one node and publishes reduced state for RViz, the web GUI and logging. `robot_state_publisher` receives the URDF and `/joint_states`, then publishes moving transforms on `/tf` and fixed transforms on `/tf_static` [8].

Frame conventions follow REP-103 (x forward, y left, z up) and the REP-105 mobile-frame separation: control owns `odom`→`base_link`, while SLAM owns `map`→`odom` [9, 10]. This matters for mapping: RTAB-Map estimates and corrects the map trajectory, and OctoMap stores probabilistic 3D occupancy as occupied, free and unknown space [13, 14, 16]. Depth and room-scan data use `sensor_msgs/PointCloud2`, whose official message definition stores an N-dimensional point collection as a binary blob described by fields and permits image-organized depth clouds [12].

The torque measurements in this document use MuJoCo’s applied actuator generalized force. MuJoCo documents actuator force transmission into `mjData.qfrc_actuator`; sampling that field gives the torque actually applied by the physics actuator at the joint DOF, after the command path and model limits, rather than only the requested impedance term [11]. RMS is then computed as a time-weighted load summary, and it is reported together with peaks, overload exposure, speed at peak and clipping. That combination is necessary because a 100 s RMS value can hide startup or touchdown spikes.

¹ROS 2 documentation source, `About-Nodes.rst`: https://github.com/ros2/ros2_documentation/blob/rolling/source/ROS-Framework/About-Nodes.rst.

2.1 Local evidence map

Claim area	Source files and artifacts
Generated rigid-body model	tools/cad_to_model.py; ros2_ws/src/robodog_description/config/robot_parameters.yaml; ros2_ws/src/robodog_description/urdf/robodog.urdf.xacro
RS06 ratings and knee belt	docs/rs06_datasheet_audit.md; ros2_ws/src/robodog_description/config/robstride06.yaml; ros2_ws/src/robodog_hardware/robodog_hardware/transmission.py
Control loop and ROS contracts	ros2_ws/src/robodog_control/robodog_control/control_node.py; ros2_ws/src/robodog_msgs/msg/*.*; ros2_ws/src/robodog_msgs/srv/*.*
Kinematics and force mapping	ros2_ws/src/robodog_control/robodog_control/kinematics.py; ros2_ws/src/robodog_control/test/test_kinematics.py
Gait and balance controller	ros2_ws/src/robodog_control/robodog_control/gait.py; ros2_ws/src/robodog_control/robodog_control/balance.py; ros2_ws/src/robodog_control/config/gaits.yaml
28 kg torque report	docs/rs06_28kg_1ms_100s.md; docs/rs06_28kg_1ms_100s.json; docs/rs06_28kg_1ms_100s.csv; tools/torque_report.py
RGB-D mapping and saved scans	docs/mapping.md; ros2_ws/src/robodog_perception/launch/mapping.launch.py; ros2_ws/src/robodog_perception/robodog_perception/map_export.py; ros2_ws/src/robodog_msgs/srv/SaveMap.srv
Primary-source research note	docs/research/robodog_scientific_foundations.md

3 Robot platform and design requirements

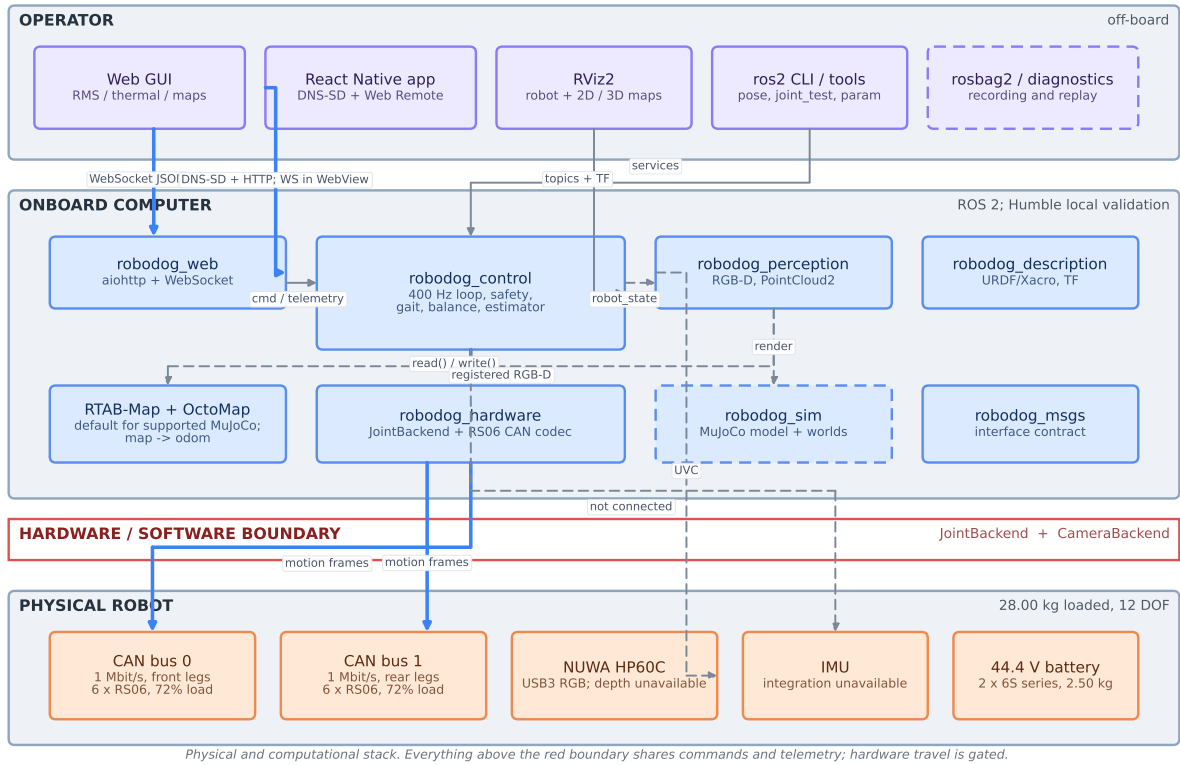


Figure 1: System architecture. The red band is the hardware/software boundary: two abstract interfaces, **JointBackend** and **CameraBackend**. Core contracts are shared; mapping is an optional extension currently supported only in MuJoCo.

3.1 Design targets

Quantity	Target	Achieved
Total mass	28.00 kg loaded	28 kg
Degrees of freedom	12 (HAA/HFE/KFE)	12
Nominal stance height	—	320 mm (77.4 % of leg reach)
Static torque margin	within continuous	requires current per-motor report
Control rate	—	400 Hz, sim and hardware alike
Actuator	ROBSTRIDE06	8/36 N·m cont/peak

3.2 Mass budget

The new CAD export totals approximately 7.549 kg. Its small placeholder motor masses are removed before adding twelve 0.621 kg RS06 actuators. The base configuration totals 19.719271 kg, including an estimated battery, electronics and mounting allowance. Two 1.250 kg battery packs contribute 2.500 kg; the twelve actuators contribute 7.452 kg. The loaded simulation adds 8.280729 kg of centered, removable test ballast for the 28 kg total. This ballast is not a CAD-derived component. The remaining component masses and their placement must be checked against the assembled machine.

Generated per-link examples are base 18.237 kg, front-left hip 908.6 g, thigh 1292.2 g and calf 202.5 g. The base-link mass center has longitudinal coordinate -3.05 mm; this is not a measurement of the whole-robot center of mass in an arbitrary pose.

3.3 Coordinate systems and conventions

Frames. REP-103 throughout: x forward, y left, z up. `base_link` is at the body geometric centre. `odom` is the world frame for the estimated base pose and the test-world markers. Camera optical frames follow the ROS optical convention (z forward, x right, y down), related to their body frames by $\text{rpy} = (-\pi/2, 0, -\pi/2)$. When mapping is active, control owns `odom`→`base_link` and RTAB-Map owns `map`→`odom`; the mapper does not replace the base-state source. ROS uses wall timestamps without a `/clock` topic.

Joint naming. `{FL,FR,RL,RR}_{haa,hfe,kfe}_joint`. HAA is hip abduction/adduction, HFE hip flexion/extension, KFE the knee.

Decision — Joint axes are not mirrored between left and right

Every HAA axis is $+x$ and every HFE and KFE axis is $+y$, expressed in `base_link` orientation, for all four legs. The alternative — mirroring the right side so that “positive” means abduction everywhere — was rejected because it makes the URDF axes differ per leg and turns every sign question into a per-leg question.

The consequence must be remembered: **positive HAA abducts the left legs and adducts the right legs**. Sim-to-real then reduces to one sign per joint in `robstride_bus.yaml`, not a frame convention.

Zero and sign. All joints at zero puts the legs straight down, feet at longitudinal offsets $\pm(h_{aa,x} + h_{fe,dx})$, lateral offsets $\pm(h_{aa,y} + h_{fe,dr} + d_{\text{thigh},y})$, and height $-(L_1 + L_2)$ relative to the base. Positive HFE swings the thigh backward. Negative KFE folds the knee backward; $\text{KFE} = 0$ is fully extended and the upper limit, so the knee cannot hyperextend.

Decision — Named poses are specified as a base height, not joint angles

Each foot hangs vertically below its own HFE axis. That one rule yields, at once: identical joint angles on all four legs despite the non-mirrored axes; a nominal support polygon of 582×289 mm. The whole-robot center of mass depends on payload placement and leg pose. Placing the feet under the *HAA* axes instead would need different front and rear angles, because the HFE sits 60 mm outboard of the HAA, and changes the support lever arms and joint utilization; compare motor-side loads through the active knee transmission.

4 Model generation and robot mathematics

4.1 CAD to URDF pipeline

The Onshape export is a flat tree of 264 part-links with 12 revolute joints, saved at an arbitrary mate pose. `tools/cad_to_model.py` turns it into a canonical 13-body model:

1. forward-kinematics every part into the root frame;
2. cut the tree at the 12 revolute joints, giving 13 rigid bodies;
3. recover the pose-invariant leg geometry (link lengths, axis offsets);
4. solve the CAD mate angles against the canonical zero definition;
5. re-express every visual mesh and inertia tensor in canonical body frames;
6. substitute RS06 mass properties for the new CAD motor placeholders.

The generation tool prints canonical-chain residuals against the active CAD. Review these together with link masses and mesh transforms after each export; a small kinematic residual alone does not verify self-collision clearance.

Decision — The URDF is generated, never hand-edited

`robot_parameters.yaml` is the single source of truth. The Xacro, the MuJoCo MJCF, the inverse kinematics and this document all read it. A hand-maintained URDF drifts from the CAD, and a drift between the model the physics uses and the model the IK uses presents as a control bug that is very hard to attribute.

4.2 Kinematics

Parameter	Value	Meaning
HAA position	$\pm 231, \pm 60$ mm	from <code>base_link</code>
HAA $\rightarrow$ HFE	60 mm longitudinal, 16 mm radial	
Thigh length L_1	192 mm	HFE to KFE, perpendicular
Thigh lateral	68.5 mm	along the HFE axis
Shank length L_2	195.62 mm	KFE to foot centre
Leg-plane offset	84.5 mm	constant, = radial + lateral
Maximum reach	387.62 mm	$L_1 + L_2$
Foot sphere	20 mm radius	contact geometry

Because rotation about y leaves y unchanged, the lateral offset collapses to a constant. That is what makes the inverse kinematics closed-form: the abduction angle is fixed by the (y, z) projection alone, and what remains is a planar two-link problem. Forward kinematics from the HAA joint frame:

$$p = R_x(q_0)(d_1 + R_y(q_1)(d_2 + R_y(q_2) d_3))$$

with $d_1 = (s_x \text{ hfe}_{dx}, s_y \text{ hfe}_{dr}, 0)$, $d_2 = (0, s_y \text{ lat}, -L_1)$, $d_3 = (0, 0, -L_2)$.

The IK returns only the knee-backward branch ($q_2 \leq 0$); the mirror solution exists mathematically but the robot cannot reach it. Round-tripping FK through IK over the full joint range gives

a worst-case position error below 10^{-9} m, and the analytic Jacobian matches central differences to 10^{-6} .

4.3 Knee drive topology

The knee actuator is mounted proximally on the upper leg and drives the knee through a 2:1 external belt. The internal 9:1 actuator gearbox is already included in the RS06 output specifications. With external ratio $r = 2$ and assumed efficiency $\eta = 0.95$,

$$q_m = r q_j, \quad \dot{q}_m = r \dot{q}_j, \quad \tau_j = r \eta \tau_m, \quad J_j = r^2 J_m.$$

The belt does not kinematically couple hip and knee angles. The gait and Jacobian use joint coordinates; the shared transmission map converts at the hardware/simulation boundary. Motor-side torque, not knee joint torque, drives current and thermal estimates. Constant efficiency approximates motoring and does not model compliance, backlash or regenerative braking.

4.4 Joint limits

Joint	Lower (rad)	Upper (rad)	Rationale
HAA	-0.8	0.8	$\pm 45.8^\circ$ abduction envelope
HFE	-1.4	2.4	thigh forward to folded
KFE	-2.6	0	no hyperextension; folds backward only

The CAD's own joint limits are not used

The Onshape export carries mate limits that are modelling artefacts — for example `dof_f10` spans $[-0.704, 0.866]$, asymmetric and different on every leg. They describe how the assembly was posed, not what the mechanism can do. The limits above are design values chosen from the mechanical envelope and should be refined once self-collision checking runs against the real geometry.

5 Actuation and electrical system

5.1 Specification and derived quantities

Quantity	Source	Value
Mass	spec	$621 \text{ g} \pm 3 \text{ g}$
Envelope	spec	$88 \text{ mm flange} \times 49 \text{ mm}$
Gear ratio	spec	9:1, machined steel
Rotating rated / peak	spec	$11 / 36 \text{ N}\cdot\text{m}$
Continuous stall	spec	$8 \text{ N}\cdot\text{m}$
Rated speed	spec	$100 \text{ rpm} = 10.47 \text{ rad/s}$
No-load speed	spec	$480 \text{ rpm} = 50.27 \text{ rad/s}$
Torque constant K_t	spec	$1.1 \text{ N}\cdot\text{m/A}_{\text{rms}}$
Line resistance	spec	$0.23 \Omega \pm 10\%$
Phase equivalent	derived	0.115Ω (equivalent wye)
Voltage	spec	48 V rated, $15\text{--}60 \text{ V}$
Encoders	spec	two magnetic encoders
Encoder resolution	unverified	not used to certify position accuracy
Rated phase current	derived	$10.11 \text{ A}_{\text{rms}}$
Copper loss at rating	derived	35.3 W
Output equivalent inertia	spec	$1.2 \times 10^{-2} \text{ kg}\cdot\text{m}^2$

Motor-side electrical model. The published output-referred $K_t = 1.1 \text{ N}\cdot\text{m}/\text{A}_{\text{rms}}$ provides a low-load phase-current estimate. Knee torque is first divided by 2×0.95 . The published 0.23Ω line resistance gives 0.115Ω under an equivalent-wye model; the latter is a modeling conversion, not a separately specified winding resistance. The constant- K_t approximation does not reproduce the published peak-current limit and is not a DC-bus current estimate or a verified Iq-to-RMS conversion. Thermal resistance and heat capacity remain uncalibrated, so steady-temperature predictions are illustrative.

Equivalent output inertia. The manufacturer specifies $0.012 \text{ kg}\cdot\text{m}^2$ at the actuator’s low-speed output. The external 2:1 knee reduction multiplies it by four to $0.048 \text{ kg}\cdot\text{m}^2$. This includes the internal drive’s equivalent inertia, not a direct bare-rotor measurement. Dividing by the internal ratio squared would only infer an equivalent rotor-side quantity. Validate assembly dynamics and friction without discarding the published output inertia.

5.2 CAN interface

Extended 29-bit identifiers at 1 Mbit/s. The identifier carries [28:24] `comm type` | [23:8] `data` | [7:0] `target`. The motion-control frame (type 1) is the MIT-style impedance command: target angle, velocity, k_p and k_d occupy the eight payload bytes, and the feed-forward torque rides in the identifier’s data field because the payload is full.

Decision — The command interface is the actuator’s native impedance frame

`JointCommand` carries (position, velocity, effort, k_p , k_d) — exactly one CAN frame. A narrower interface, such as position only, would force the host to close a loop the motor already closes in firmware at tens of kHz. The host sets *setpoints*; it is not the servo.

5.3 Bus budget

An extended-ID frame with eight data bytes is 131 bits nominal and up to 159 with worst-case stuffing; 150 bits is the design figure. Each joint needs one command and one feedback frame per cycle.

Buses	Motors/bus	Rate	Load	Verdict
1	12	250 Hz	90 %	too tight, no margin
1	12	400 Hz	144 %	impossible
2	6	400 Hz	72 %	selected
2	6	500 Hz	90 %	rejected, no margin
3	4	500 Hz	60 %	upgrade path

Decision — Two buses at 400 Hz, split front/rear

400 Hz is not a compromise on control quality: because the RS06 closes its position and velocity loop in firmware, the host rate is a *setpoint update* rate. The same rate is used in simulation so that gains tuned in MuJoCo transfer without change.

The split is front/rear rather than left/right so that each bus carries a complete leg *pair*: a single bus failure leaves the robot in a recognisable posture rather than a diagonal collapse. `RobStride06Backend.check_bus_budget()` refuses any configuration above 80 % at start-up.

6 Locomotion and trajectory generation

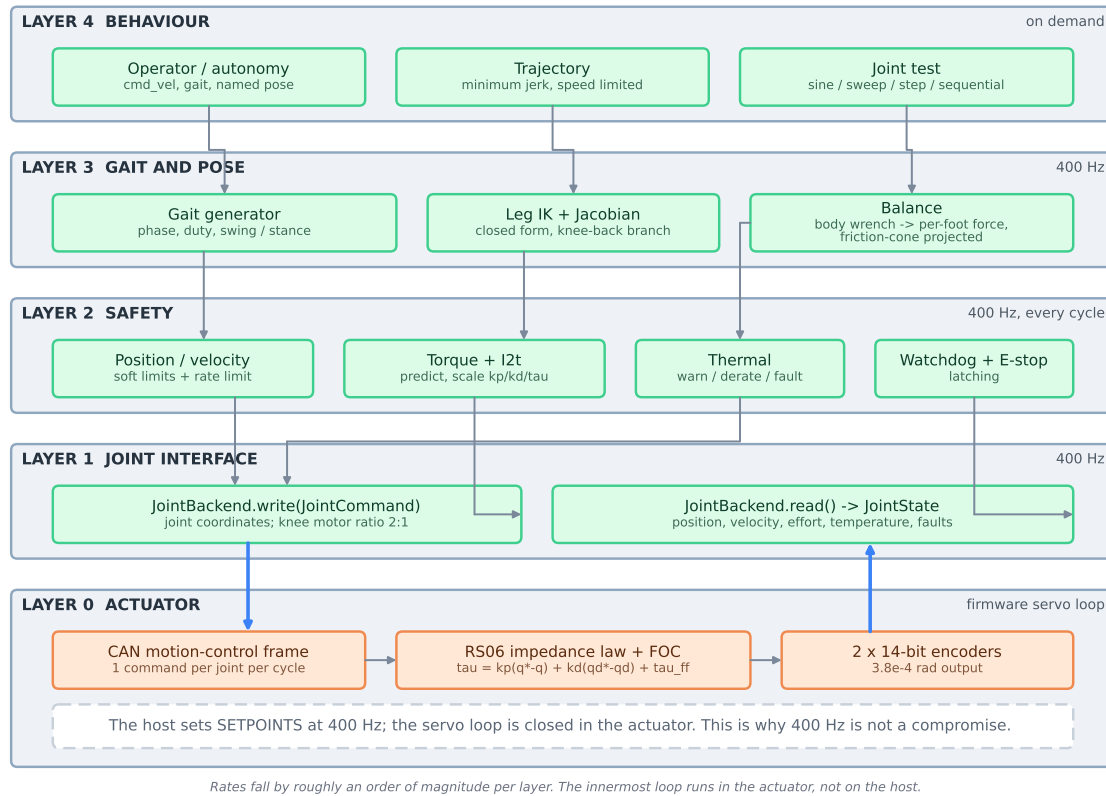


Figure 2: Control layers. Rates fall by roughly an order of magnitude per layer, and the innermost loop is not on the host at all.

6.1 The loop

One thread, 400 Hz, in `robodog_control_node`:

`read()` → estimate → active controller → safety → `write()` → `step()`

ROS callbacks run on a separate executor and only ever set fields the loop reads; nothing in a callback touches the backend. Publishing is decimated — `RobotState` at 50 Hz, `/joint_states` at 100 Hz — because publishing telemetry at 400 Hz would cost more than the control it reports on.

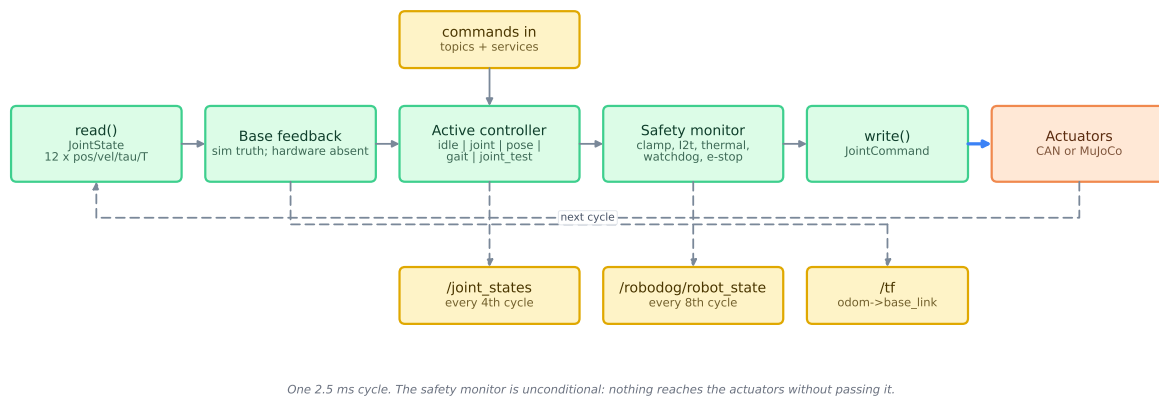


Figure 3: One 400 Hz cycle. The safety monitor is unconditional.

Decision — Controllers are selected, not composed

Exactly one of {idle, joint, pose, gait, joint_test} drives the joints at any moment. Making that exclusive by construction is worth more than the flexibility of a controller chain, because two things writing to the same actuator is a failure mode with no good recovery.

6.2 Impedance control

The actuator evaluates

$$\tau = k_p (q^* - q) + k_d (\dot{q}^* - \dot{q}) + \tau_{ff}$$

in firmware. Effective joint inertia includes the published equivalent actuator output inertia, reflected through the external belt, plus configuration-dependent link inertia. For an identified effective inertia J , the local linear estimates are $\omega_n = \sqrt{k_p/J}$ and $k_{d,critical} = 2\sqrt{k_p J}$. The knee's actuator contribution is four times that of a direct-drive hip, so one old RS02 inertia calculation cannot characterize every current joint.

Context	k_p	k_d	Rationale
Named pose, joint test	80	2.0	moderate, unloaded
Stance leg	90	2.0	feedback around balance feed-forward
Swing leg	60	1.5	tracks the unloaded trajectory

6.3 Gravity feed-forward and the force convention

Each stance leg is fed the torque that holds its share of the body weight. The sign convention is stated once and used everywhere:

f is the force the *environment* applies to the foot (the ground reaction, $+z$ in stance), so

$$\tau_{actuator} = -J^T f, \quad f = -(J^T)^{-1} \tau_{actuator}.$$

This sign is not cosmetic

An inverted feed-forward makes a stance leg push the body *down*. Measured in MuJoCo: with correct feed-forward the body sags 0.31 mm below the commanded 320 mm; without feed-forward it sags 5.34 mm; with the sign inverted it sags 11.95 mm. Magnitude-only tests pass in all three cases, which is why `test_kinematics.py` asserts the *direction* of the knee torque explicitly.

6.4 Gait generation

A single normalised cycle phase advances at the step frequency. Each leg has a fixed offset and is in stance for `duty_factor` of its period.

Gait	Offsets	Duty	Freq (Hz)	Character
stand	—	1.00	0.0	all feet planted
walk	0, 0.50, 0.75, 0.25	0.78	1.5	one foot up, brief full support
trot	0, 0.50, 0.50, 0	0.50	2.2	diagonal pairs; default travel
pace	0, 0.50, 0, 0.50	0.50	2.0	lateral pairs; rolls this narrow body
bound	0, 0, 0.50, 0.50	0.45	2.6	front then rear; most demanding

6.4.1 What the generator actually does

The gait generator is a *foot trajectory planner*. It does not compute torques and it does not know what a motor is. Once per control cycle it is given the elapsed time and the body's measured state, and it returns, for each of the four legs, where that foot should be, how fast it should be going, and how hard it should push. Everything else — inverse kinematics, the impedance law, the actuator — happens downstream.

Concretely, `GaitGenerator.update(dt, feedback)` returns twelve joint positions, twelve joint velocities, twelve feed-forward torques, and a four-element contact flag. The joint values come from running each foot position through the closed-form leg IK; the torques come from the balance layer. A gait is therefore defined entirely by *geometry and timing*, and adding one is a block in `gaits.yaml` plus a phase offset — no new code.

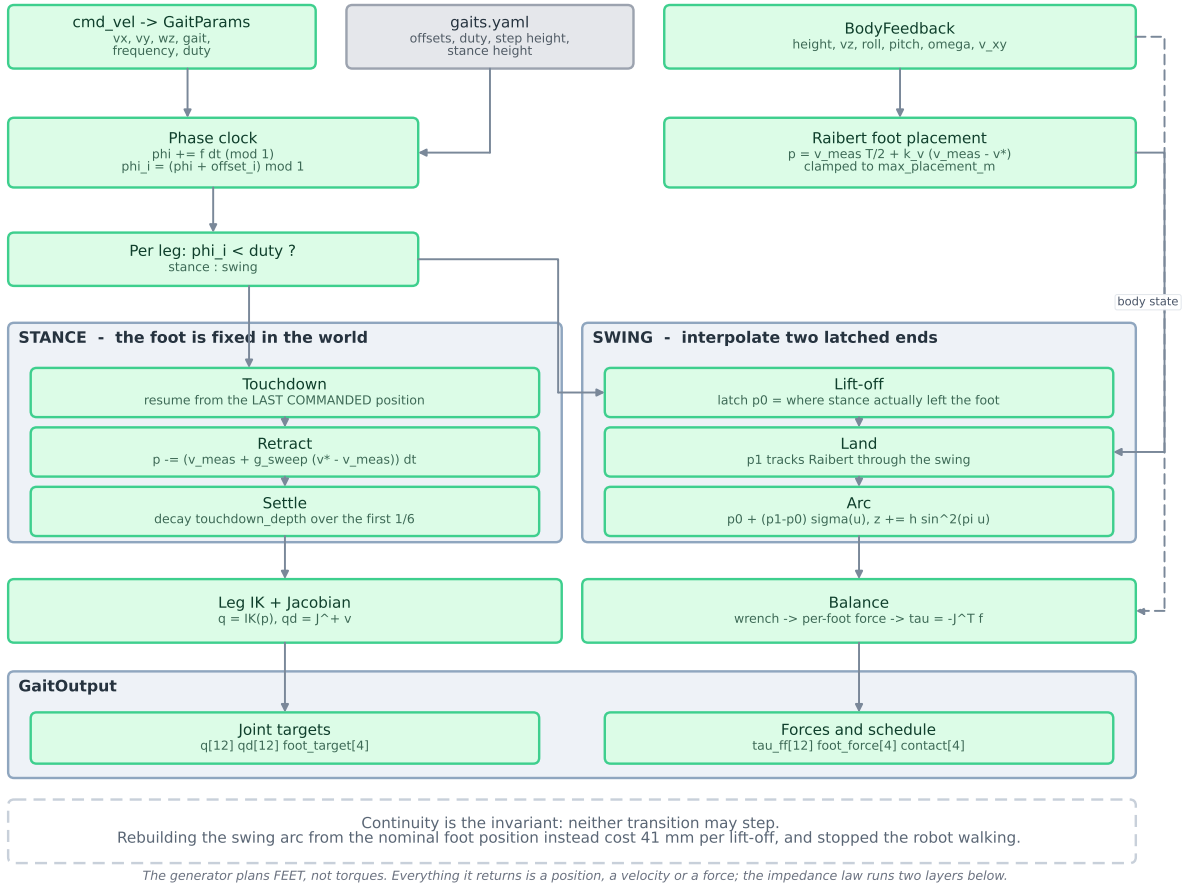


Figure 4: One control cycle of the gait generator. The generator plans feet, not torques: everything it returns is a position, a velocity or a force, and the impedance law runs two layers below. Continuity is the invariant — stance resumes from the last commanded position and swing starts from where stance left the foot, so the command cannot step at either transition.

6.4.2 The phase model

One normalised phase $\phi \in [0, 1)$ advances at the step frequency and wraps. Each leg i reads the same clock through its own offset:

$$\phi_i = (\phi + \text{offset}_i) \bmod 1, \quad \text{leg } i \text{ is in STANCE iff } \phi_i < D$$

where D is the duty factor. That single line is what distinguishes a walk from a trot: the offsets decide which feet are down together, and D decides for how long. A duty of 0.75 with the walk

offsets always leaves three feet down, so the robot is statically stable; a duty of 0.5 with the trot offsets leaves two, on a diagonal, so it is not.

6.4.3 Stance: the foot is a fixed point in the world

A planted foot does not move. It follows that in the *body* frame it must translate backwards at exactly the body’s velocity — if the body advances 10 mm, the foot must appear 10 mm further back, or the command is asking the leg to drag the robot through the contact.

$$\mathbf{p}_i^{\text{stance}} = \mathbf{v} \delta t, \quad \mathbf{v} = \mathbf{v}_{\text{meas}} + g_{\text{sweep}} (\mathbf{v}^* - \mathbf{v}_{\text{meas}})$$

The sweep gain g_{sweep} chooses which velocity to believe, and the default of 1.0 — the *commanded* velocity — is the least obvious decision in the whole generator. It looks like it must scuff. In steady state it does not: once the body is travelling at the commanded speed, the target sweeps at exactly the rate the planted foot is already moving through the body frame, and the relative motion is zero. When the body is *not* at speed, the target runs ahead of the foot and the leg impedance turns that gap into a tangential force.

That is the point. Sweeping at the commanded velocity is proportional velocity feedback routed through the leg, and it is the dominant thing regulating speed. Sweeping at the measured velocity instead is self-consistent at any speed, including the wrong one: measured at $g_{\text{sweep}} = 0$, a walk commanded to 0.15 m/s free-ran to 0.272 m/s and then fell over.

6.4.4 Swing: interpolate between two latched endpoints

Swing carries the foot from where it left the ground to where it should next land. Both endpoints are explicit positions:

- $\mathbf{p}_0$ is latched at lift-off — wherever stance actually left the foot, not where it was supposed to be;
- $\mathbf{p}_1$ is the landing point, tracked continuously through the swing so a foot in the air still reacts to the body accelerating.

$$\mathbf{p}^{\text{swing}}(u) = \mathbf{p}_0 + (\mathbf{p}_1 - \mathbf{p}_0) \sigma(u), \quad \sigma(u) = u - \frac{\sin 2\pi u}{2\pi}$$

$\sigma'(u) = 1 - \cos 2\pi u$ vanishes at both ends, so the foot leaves and arrives with no horizontal velocity relative to the ground — it is placed, not dragged. The vertical profile $\sin^2(\pi u)$ is likewise zero-slope at both ends, with a small `touchdown_depth` subtracted late in swing so the foot seeks the floor rather than hovering over it, and decayed again over the first sixth of stance so that seeking does not itself become a step.

Interpolating between two latched endpoints is what makes the command continuous, and it is worth being explicit about why that is not a detail. Section 6.6 records what happened when it was not.

6.4.5 Foot placement: where velocity is really decided

Attitude control holds a robot up; it cannot make it go anywhere. A legged robot regulates velocity by *where it puts its feet*. The landing point uses the Raibert heuristic:

$$\mathbf{p}_1 = \mathbf{p}_{\text{nom}} + \underbrace{\mathbf{v}_{\text{meas}} \frac{T_{\text{stance}}}{2}}_{\text{neutral point}} + \underbrace{k_v (\mathbf{v}_{\text{meas}} - \mathbf{v}^*)}_{\text{correction}}$$

The first term is the landing point that *sustains* the current velocity: land half a stance-sweep ahead, sweep back a full stance-sweep, end half behind, and the stride is symmetric about the hip. The second term breaks that symmetry deliberately — landing further ahead than neutral decelerates, further behind accelerates. The offset is clamped to `max_placement_m` so a large velocity error cannot ask for a footfall outside the workspace.

6.4.6 Three things regulate speed, and they are not interchangeable

Mechanism	Acts through	Role
Stance sweep	leg impedance	Dominant. Turns velocity error into tangential force every cycle.
Tangential wrench	feed-forward torque	Explicit force request, $k_{p,v}(\mathbf{v}^* - \mathbf{v})$, friction-cone limited.
Foot placement	next footfall	Sets the velocity the gait is stable <i>at</i> ; acts once per step, not continuously.

Removing any one degrades tracking; removing the first breaks it outright.

6.4.7 Force distribution

Positions alone do not hold a robot up. The contact flag from the phase model tells the balance layer which feet may push, and it solves for the ground reaction at each of them that best reproduces the desired body wrench (Section 6.5). Those forces become feed-forward joint torques through $\boldsymbol{\tau} = -J^\top \mathbf{f}$. The gait generator supplies the geometry and the contact schedule; the balance layer supplies the forces; neither one alone walks.

6.4.8 Per-cycle summary

1. Advance ϕ ; for each leg compute ϕ_i and its stance/swing state.
2. Compute the Raibert landing offset from measured and commanded velocity.
3. STANCE: on touchdown, resume from the last commanded position; retract at the sweep velocity; decay the touchdown depth.
4. SWING: on lift-off, latch $\mathbf{p}_0$; interpolate $\mathbf{p}_0 \rightarrow \mathbf{p}_1$ with the zero-endpoint-velocity profile and the $\sin^2$ lift.
5. Run each foot position and velocity through the leg IK and Jacobian to get q and $\dot{q}$.
6. Ask the balance layer for the body wrench, distribute it over the stance feet, convert to feed-forward torque.
7. Return q , $\dot{q}$, τ_{ff} , contact.

Stride length is never configured, and there is no stride parameter to configure. Swing interpolates between where the foot left the ground and where placement says it should land, so the stride is whatever those two points imply. The workspace cap is `max_placement_m`, which bounds the landing offset directly — 0.12 m against a 0.43 m leg, comfortably inside reach.

6.5 Balance

A foot-trajectory generator alone does not walk. With the open-loop gait and no balance term, a 0.3 m/s trot in MuJoCo reached 25° of body tilt, saturated the actuators at the 36 N·m peak and travelled 28 mm in six seconds. The feet were following their commanded paths; the body was not being held up. `balance.py` adds the two things that were missing.

Force distribution. A PD controller on height, roll, pitch, yaw rate and horizontal velocity produces a desired body wrench, and the stance foot forces are solved to reproduce it:

$$\sum_i \mathbf{f}_i = \mathbf{F}, \quad \sum_i \mathbf{r}_i \times \mathbf{f}_i = \mathbf{M}$$

Six equations in $3n_{\text{stance}}$ unknowns — exactly determined for a trot, under-determined for a walk or a full stance — solved by least squares, then projected into what a contact can deliver: $f_z \geq 0$ and $\|\mathbf{f}_{xy}\| \leq \mu f_z$.

Decision — A stance leg is force controlled, not position controlled

This was measured the hard way. With `stance_kp` at 120 the stance legs showed 0.17rad of tracking error and 20 N·m of pure disagreement with reality: the commanded foot position was advancing at the *commanded* body velocity while the foot was physically planted, so the leg was dragging the robot through its own contact. Two changes follow. The stance foot now translates at a blend of the *measured* and commanded velocity (`stance_sweep_gain`), and the placement offset is latched at touchdown so the command cannot move under a planted foot. Stiffness is reduced to 90: raising it turns the leg into a spring that anchors the body to its own foot and cancels the propulsion force.

Foot placement. Attitude control cannot regulate velocity — a legged robot steers by where it puts its feet. The Raibert heuristic places the landing point at $v T_{\text{stance}}/2 + k_v(v - v^*)$.

6.6 Historical RS02 locomotion performance

Six seconds of each gait from the standing keyframe, commanded in a straight line, with the balance layer active. Drift is the largest lateral excursion.

Gait	v_x^* (m/s)	actual (m/s)	error (m/s)	tilt (deg)	drift (m)	Verdict
stand	0.00	−0.001	−0.001	0.1	0.000	solid
walk	0.15	+0.155	+0.005	7.4	0.024	tracks
walk	0.30	+0.300	+0.000	10.9	0.040	tracks
trot	0.30	+0.297	−0.003	5.1	0.040	tracks; default
trot	0.50	+0.493	−0.007	7.2	0.054	tracks
pace	0.30	+0.146	−0.154	16.6	0.066	rolls; comparison only
bound	0.30	+0.193	−0.107	18.4	0.001	probe gait only

Walk and trot — the two gaits intended for travel — track commanded velocity to within 7 mm/s. Pace and bound do not, which is consistent with what they are for: pace rolls a body only 120 mm wide between hip pivots and is kept for comparison, and bound exists to probe the peak-torque and I^2t limits in simulation before hardware ever sees them.

6.6.1 How it was broken, and what that cost

For most of this project the robot did not travel. It stepped in place and drifted backwards at roughly 0.04 m/s, while every travelling gait drew 141 to 201 % of the continuous torque rating. Both symptoms had the same two causes, and neither was in the balance layer or the gains, which is where the effort went first.

A 41 mm step at every lift-off. Stance retracts the foot at the sweep velocity, but the swing arc was rebuilt from scratch each cycle as $\mathbf{p}_{\text{nom}} + \text{stride}(\sigma - \frac{1}{2})$ — which assumes stance had swept the foot back by the full *commanded* stride. Whenever the robot was not already at the commanded speed the two disagreed, and the difference was commanded in a single 2.5 ms tick at every lift-off: 41 mm from a standstill at 0.30 m/s. That step drove 0.33rad of tracking error, which saturated the hips at the 36 N·m peak and so destroyed the force distribution the balance layer had just computed. The robot could not accelerate, so the measured velocity stayed at zero, so the step never shrank. The gait fought itself into a steady state.

Two jammed hips. The thigh capsule and the HAA actuator cylinder are both conservative bounding volumes around parts that clear each other in the CAD, and MuJoCo does not auto-filter the pair because the thigh is the base’s *grandchild* (base → hip → thigh), not its child. The front hips therefore jammed 13 mrad above the standing pose and would not move under 16.6 N·m, while the rear hips swung 1.59 rad on 3 N·m. The front legs could only lift with the knee, and the hip absorbed the entire impedance error as heat. The robot was, in the most literal sense, walking with two of its twelve joints welded shut.

A careful measurement of a broken system is still a measurement of a broken system

The stance-height study in `gaits.yaml` was repeated at four heights, was internally consistent, and was wrong — it concluded that the balance layer stopped helping above 340 mm, which was an artifact of the jam. So was the finding that lower stance impedance reduced torque: with the hips free, the original `stance_kp` of 90 turned out to be the best value tested. The tuning was never the problem, and no amount of it would have helped.

What eventually found both bugs was not more tuning but per-joint instrumentation: printing commanded against measured position for one leg through one gait cycle showed a 40 mm jump in the target at lift-off, and a hip angle that did not change by 0.001 rad across four seconds under peak torque. A joint that will not move under 16 N·m is not a control problem.

Both are now covered by tests that fail if they return: `test_the_foot_command_never_jumps` asserts the commanded foot position never moves more than 5 mm in one tick, at four different measured velocities, and `test_every_hip_can_swing_through_its_commanded_range` drives each hip alone and requires all four to move, and to move alike.

6.6.2 What the balance layer is for, now

Fixing the trajectory changed the answer. The balance layer used to be what kept a 0.3 m/s trot from reaching 25° of tilt; with the swing arc continuous and the hips free, the open-loop gait is well behaved on its own and the two are within a few tenths of a degree — 5.1° with balance against 4.9° without. Tilt is no longer where the layer earns its place.

Course holding is. Over six seconds at 0.5 m/s the lateral drift is 0.054 m with balance and 0.299 m without, because the Raibert term is the only thing steering the feet back under the body, and velocity tracking degrades from −0.007 to −0.018 m/s. A robot that wanders 0.3 m sideways in six seconds does not get through a doorway. The regression test asserts those two things rather than tilt.

6.7 Historical RS02 thermal load

The following table used the old RS02 model with 6 N·m continuous and 17 N·m peak ratings. RMS torque tracks copper heating, but the temperatures were estimates rather than winding measurements. Six-second runs discarded the startup transient and reported the busiest joint:

Gait	peak (N·m)	p99 (N·m)	RMS (N·m)	RMS / cont.	T_∞ (°C)	Verdict
stand	3.9	3.9	3.9	65 %	45	sustainable
walk 0.15	17.0	9.6	5.5	91 %	70	sustainable
trot 0.30	14.3	8.7	5.4	90 %	69	sustainable
trot 0.50	14.5	9.3	5.5	92 %	71	sustainable
bound 0.30	17.0	16.7	7.9	131 %	123	over limit

These supersede an earlier table in which every travelling gait sat between 141 and 201 % of continuous, implying winding temperatures from 139 to 260 °C. That table was correct about a robot with two welded hips and a discontinuous swing arc.

The interpretation was correct then and remains the useful one: the actuators were never undersized. Standing costs 3.9 N·m RMS, and steady walking at 0.5 m/s needs about 49 W of net mechanical work against 754 W rated — some 7 % — because almost all the torque a quadruped spends is holding itself up, which does no work at all. A travelling gait that costs three times the RMS of standing still is not doing three times the work; it is wasting the difference. With the waste removed, trot sits at 90 % of continuous and 69 °C, inside the 70 °C warning threshold.

Bound remains over limit, which is what it is for: it is the gait used to probe the peak-torque and I^2t path in simulation, and it should not be commanded on hardware. Pace is not listed because it does not track velocity well enough for the number to mean anything.

Margin, not comfort

90 % of continuous is passing, not roomy. Trot buys its margin partly by standing 340 mm tall rather than 320 — a straighter leg has a shorter moment arm — and that costs 20 mm of the leg travel otherwise available for terrain and attitude correction. Walk stays at 320 mm, where it is 91 %, because it is the gait for stairs and narrow passages and wants the travel. Those historical stance heights have been replaced by the current RS06 tuning and should not be copied from this table.

None of this has been validated against hardware. The thermal model is a first-order estimate from datasheet resistance and an assumed thermal resistance; the friction and damping are estimates. Treat 90 % as “plausibly inside the rating”, not as a measurement of a real motor.

6.8 State estimation

Decision — A complementary filter and leg odometry, not an EKF

The estimator module implements the intended sensor-fusion algorithm, but its real IMU input is not integrated into the control loop. Hardware travel is blocked while base feedback is unavailable. Given actual sensor input, attitude comes from blending gyro integration with the gravity direction, with the gravity term weighted by how closely the measured acceleration magnitude resembles g — during a hard push-off it does not. Height and velocity come from treating each foot in contact as a temporary fixed point.

An EKF was rejected *for now* because until the robot exists its process and measurement covariances would be invented numbers. This is honest, debuggable; it still requires sensor integration and physical validation. Yaw is not observable from an IMU without a magnetometer and is flagged as drifting.

7 Safety and load protection

7.1 Two-tier limits

Quantity	Hard (URDF)	Operational	Enforced by
Torque	36 N·m	8 N·m continuous	I^2t integrator
Velocity	50.27 rad/s	20 rad/s	clamp
Position	joint end stops	end stops inset 0.05 rad	clamp + rate limit
Temperature policy	—	70/85/100 °C	derate, then latch

Torque and speed values in this table are actuator-output references. The knee joint’s continuous budget is 15.2 N·m and peak is 68.4 N·m with the assumed 2:1 ratio and 95 % efficiency; its speed limits halve. Enforcement uses these per-joint mappings rather than one scalar joint limit.

The URDF carries modeled hard bounds, not a guarantee of sustained physical output; the operational tier constrains requests within the configured policy. Temperature thresholds are project policy and do not identify the feedback sensor as a winding-temperature measurement.

7.2 Design rules

1. **It runs in simulation too.** The monitor executes on every cycle regardless of backend. A safety layer that only activates on hardware has never been tested by the time it matters.
2. **It is a pure function.** (measurement, request, state) $\rightarrow$ (allowed command, status). No ROS, no I/O, no threads — which is what makes the whole envelope testable. `test_safety.py` is the specification.
3. **It clamps rather than rejects.** Dropping a cycle mid-stride is more dangerous than saturating one.
4. **Protective faults latch.** Clearing requires the condition to have gone away *and* an explicit operator action.

7.3 Torque limiting under impedance control

Decision — Scale the whole (k_p, k_d, τ_{ff}) triple

The motor computes $k_p e + k_d \dot{e} + \tau_{ff}$ itself, so clamping only the `effort` field would not limit the torque at all. The monitor predicts the resulting torque and, if it exceeds the budget, scales all three terms by the same factor — limiting the magnitude while preserving the commanded impedance direction.

The budget is the continuous rating normally, rising to peak while $I^2 t$ credit remains. The integrator charges as $(\tau/\tau_{cont})^2 - 1$ above the continuous reference, and discharges when below. This is a project protection heuristic, not the manufacturer’s exact overload curve or per-phase stall heating model. The published one-second 36 N·m stall limit is not an indefinitely repeatable pulse allowance.

7.4 Thermal model

$$P_{cu} = 3I_{rms}^2 R_\phi, \quad I_{rms} = \tau_m / K_t, \quad C_{th} \dot{T} = P_{cu} - \frac{T - T_{amb}}{R_{th}}$$

Here τ_m is actuator-output torque before the external belt; knee joint torque is divided by 2×0.95 . The phase equivalent follows from the published line resistance under an equivalent-wye assumption; thermal resistance and capacitance remain uncalibrated. The equation uses $R_{th} = 1.31$ K/W and $C_{th} = 310$ J/K, giving a time constant of 6.8 minutes. It runs in two roles: in simulation it synthesises a plausible temperature, because MuJoCo has no concept of copper loss and the safety architecture must be exercised; on the real robot it runs as an *observer* alongside the reported temperature, and the monitor takes the maximum of the two. The feedback temperature is not an identified winding measurement; sensor location and lag must be checked on delivered hardware. Taking the maximum does not make an uncalibrated observer a verified protection model. This single lump does not reproduce uneven per-phase heating with a stationary current vector at stall.

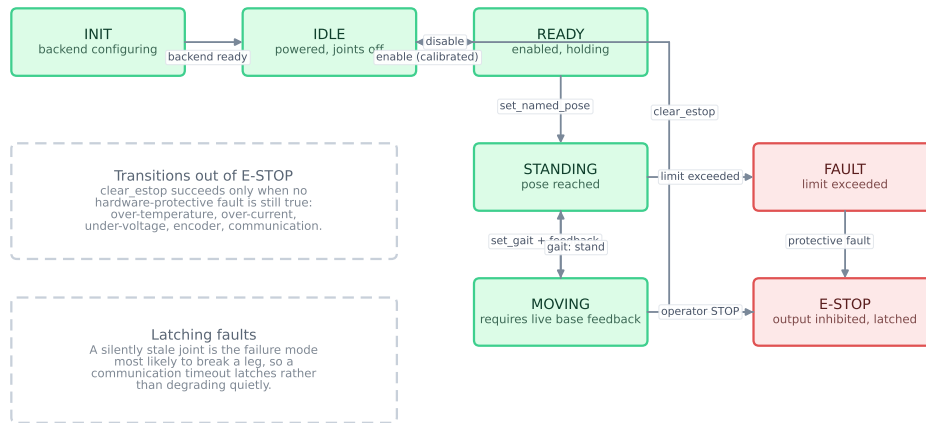
Between 85 and 100 °C the torque budget is scaled down linearly rather than cut at a threshold, because cutting output abruptly would drop the robot.

7.5 Watchdog and emergency stop

Decision — A stale command holds position; it does not go limp

Losing torque mid-stance drops a 28 kg loaded robot. The watchdog substitutes an impedance command that holds the measured position at moderate gain. If a joint is measured outside its soft range, the hold target is the limit, so the watchdog recovers the envelope gently rather than freezing outside it.

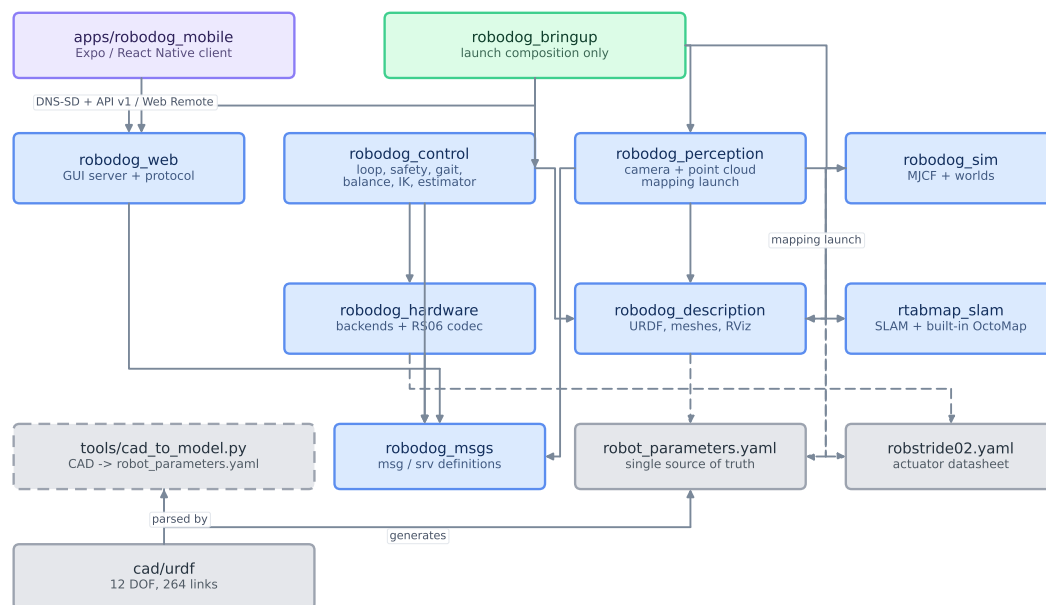
A communication timeout on any joint latches an e-stop: a silently stale joint is the failure mode most likely to break a leg.



FAULT and E-STOP are reachable from every state. Leaving E-STOP requires the cause to have cleared.

Figure 5: Robot states. FAULT and E-STOP are reachable from everywhere; leaving E-STOP requires the cause to have cleared.

8 ROS 2 software and data architecture



ROS 2 package graph. Arrows point from dependant to dependency. RTAB-Map runs by default for the supported MuJoCo RGB-D pipeline.

Figure 6: ROS 2 package graph. Arrows point from dependant to dependency. The joint backend isolates control from the physics engine; launch composition selects simulation assets and mapping support.

8.1 Packages

Package	Build	Responsibility
robodog_msgs	ament_cmake	message and service contract
robodog_description	ament_cmake	URDF/Xacro, meshes, RViz, actuator data
robodog_hardware	ament_python	JointBackend + RS06 CAN codec
robodog_control	ament_python	control loop, safety, gait, IK, estimator
robodog_sim	ament_python	MJCF generator, test worlds, markers
robodog_perception	ament_python	camera, depth, cloud, mapping launch
robodog_web	ament_python	GUI server, protocol, rolling diagnostics
robodog_bringup	ament_cmake	launch composition only
apps/robodog_mobile	Expo/TypeScript	phone/tablet discovery, status and control shell

`robodog_msgs` depends on nothing but `std_msgs` and `geometry_msgs`: it is the contract between layers and must not acquire an implementation dependency.

8.2 Package-level architecture in depth

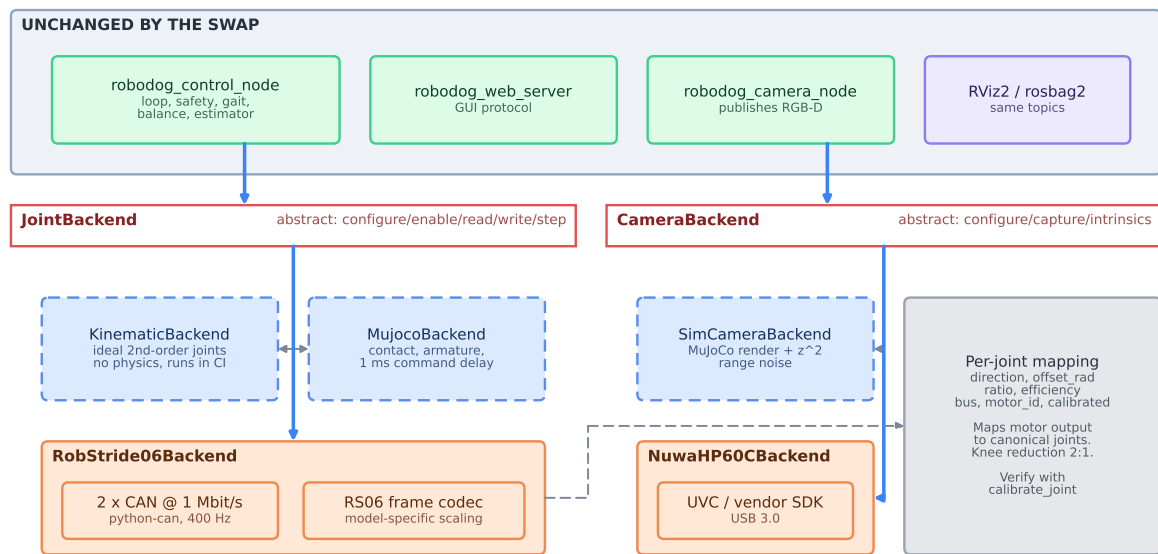
The workspace is deliberately split by ownership boundary rather than by file type. `robodog_msgs` is the only package allowed to be imported by every layer. Its aggregate `RobotState` message carries the full coherent robot snapshot, while services such as `SetGait`, `EnableJoints`, `EmergencyStop` and `SaveMap` describe the small set of state-changing operations exposed outside the controller. This keeps GUI, logger and mobile code from depending on Python implementation classes.

`robodog_description` owns static robot facts: generated Xacro, meshes, actuator YAML, named poses, joint limits and mass budget. `robodog_sim` converts the same description into MJCF and world geometry, then supplies the MuJoCo backend and RViz world markers. `robodog_hardware`

owns motor/joint conversions, RS06 CAN framing, backend registration and telemetry types. `robodog_control` owns every command decision that can move a joint: the gait generator, balance law, state estimator, safety monitor and single control loop. `robodog_perception` owns camera backends, PointCloud2 generation, mapping launch, RTAB-Map database/session planning and room-scan export. `robodog_web` bridges the ROS graph into a constrained browser protocol. `robodog_bringup` only composes these pieces.

The practical rule is simple: a lower layer may expose facts and bounded operations, but it may not reach upward. For example, the map exporter can call RTAB-Map backup and OctoMap saver services, but it cannot change gait state; the GUI can request `/robodog/mapping/save_map`, but map export cannot write directly into controller internals. This is why the same launch file can turn mapping on for MuJoCo and leave it disabled for hardware without changing the controller.

8.3 The hardware boundary



Replacing simulation with hardware is a launch argument. Hardware startup stays disabled; travel requires live base feedback.

Figure 7: The command and telemetry contracts survive the swap to hardware. Hardware sensor integration remains outstanding; optional mapping is disabled there. The direction and offset map translates motor and canonical joint coordinates.

`JointBackend` declares configuration, command, feedback and base-state methods:

```
class JointBackend(ABC):
    name: str; is_simulation: bool
    def configure(self) -> None           # open devices / load model
    def enable(self, mask) -> None        # energise
    def disable(self, mask) -> None       # de-energise, back-driveable
    def read(self) -> JointState          # pos, vel, effort, temperature, faults
    def write(self, cmd: JointCommand)    # already safety-clamped
    def step(self, dt) -> None            # advance a simulator; no-op on
        hardware                          # None on real hardware
    def base_state(self) -> BaseState | None
```

Decision — `base_state()` returns `None` on the real robot

A simulator knows the floating-base state exactly; the real robot has no sensor for absolute base pose. Rather than inventing one, the contract makes the absence explicit and forces every consumer to handle it. Real IMU integration is currently missing, so hardware travel is refused rather than synthesizing gravity and zero gyro samples. The estimator design below describes a future integration path; it does not establish a working hardware balance loop.

Three implementations exist. `KinematicBackend` models each joint as an independent second-order system with the reflected rotor inertia — no physics engine, no contact, no gravity — so core control can be exercised without a physics engine. Mapping is disabled in this mode. `MujocoBackend` adds rigid-body dynamics and contact. `RobStride06Backend` drives the real motors.

8.4 Interfaces

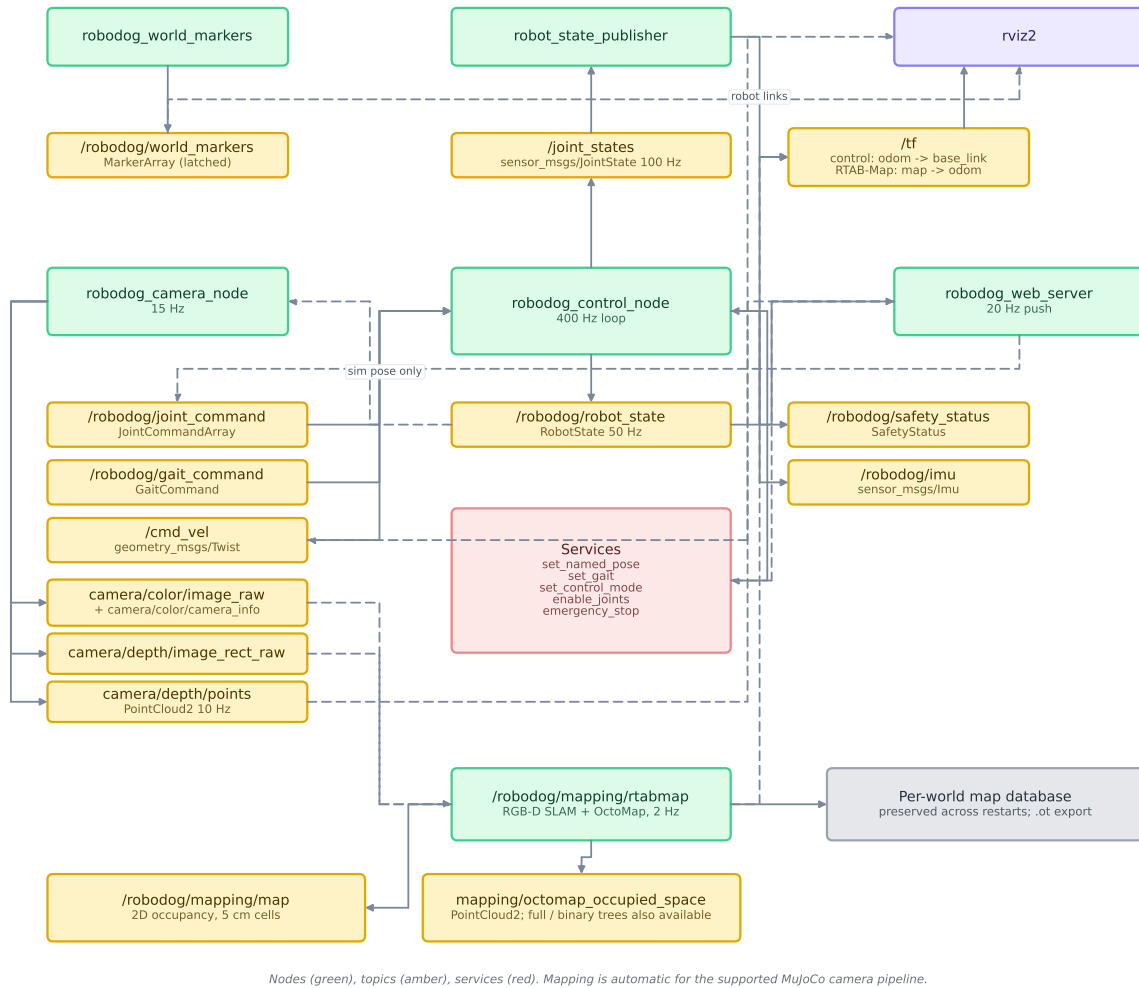


Figure 8: Core node and topic graph. The supported MuJoCo camera pipeline additionally launches RTAB-Map by default; hardware depth and odometry integration remain prerequisites for enabling mapping there.

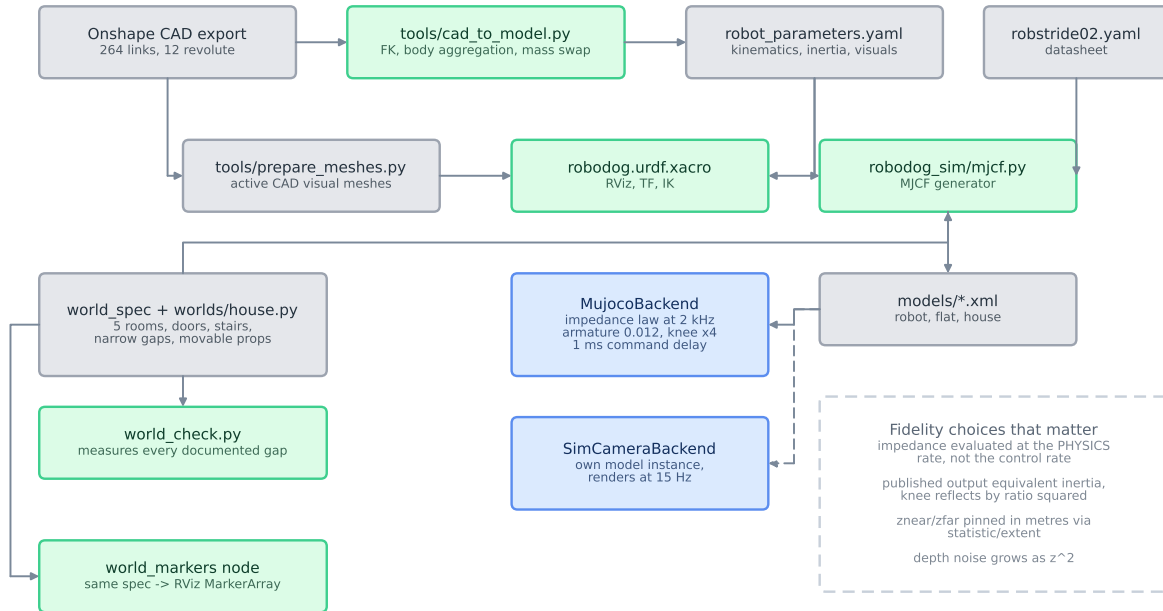
Topic	Type	Rate	Purpose
/joint_states	JointState	100	TF, RViz, standard tooling
/robodog/robot_state	RobotState	50	aggregate telemetry
/robodog/safety_status	SafetyStatus	50	limits, faults, loop health
/robodog/imu	Imu	50	attitude and rates
/robodog/joint_command	JointCommandArray	—	direct joint control
/robodog/gait_command	GaitCommand	—	gait selection
/cmd_vel	Twist	—	body velocity
/robodog/world_markers	MarkerArray	latched	test environment
camera/color/image_raw	Image (rgb8)	15	colour
camera/depth/image_rect_raw	Image (16UC1)	15	depth, millimetres
camera/depth/points	PointCloud2	10	range-filtered cloud

Services: set_named_pose, set_gait, set_control_mode, enable_joints, emergency_stop.

Decision — One aggregate telemetry message

RobotState carries all twelve joints, the four feet, the base estimate, controller state, safety state and simulation state in one message. A consumer therefore never has to time-align several topics to render a consistent picture. /joint_states is published separately and at a different rate because the wider ROS ecosystem expects it.

9 Simulation model and test environments



The MJCF and the URDF are generated from the same parameters, so the physics and the kinematics cannot drift apart.

Figure 9: The MJCF and the URDF are generated from the same parameters, so the physics and the kinematics cannot drift apart.

9.1 Fidelity choices

These are not convenience settings; each is here because getting it wrong yields a controller that works in simulation and fails on the robot.

1. **The impedance law is evaluated at the physics rate**, not the control rate. The real RS06 closes its loop in firmware at tens of kHz; holding a 400 Hz zero-order hold on $k_p(q^* - q)$ would make the simulated joints noticeably softer than the hardware and would tune gains that are wrong on the robot. Only the *setpoint* is held.
2. **armature carries equivalent output inertia**: $0.012 \text{ kg}\cdot\text{m}^2$ for direct hips and $0.048 \text{ kg}\cdot\text{m}^2$ after the external knee reduction. Omitting it overstates acceleration.
3. **Command latency is injected** — two physics steps, about 1 ms, the order of a 1 Mbit/s bus at 400 Hz. Controllers tuned against a zero-latency plant are optimistic in the wrong direction.
4. **Capsules, not cylinders**, for the leg segments. URDF has no capsule primitive; MuJoCo does, and capsule contact is far better conditioned than the flat-ended cylinder a URDF must approximate it with.
5. **Timestep 0.5 ms** (2 kHz), exactly five substeps per control cycle. A non-integer ratio would make the effective command latency jitter.

Not modelled: gearbox backlash, belt compliance in the knee drive, current-loop bandwidth, and CAN jitter.

9.2 Validation against the model

The current tests compare generated URDF and MJCF masses, origins, axes and joint limits and verify unobstructed hip motion. At rest, total ground reaction should approach mg ; for the loaded 28.000 kg model that is approximately 274.68 N. Equal load sharing would be about 68.67 N per foot, but the actual front/rear split depends on the generated center of mass and stance. Current simulated displacement, tilt and per-motor loads are recorded in [docs/rs06_28kg_1ms_100s.md](#), with JSON, CSV and MuJoCo XML exports; [the locomotion validation report](#) summarizes failed speeds, turn-rate errors and the shared-controller Go2 comparison. The earlier 10 kg validation numbers are not evidence for this configuration.

9.3 Test environment

Decision — One world description, two renderers

`world_spec.py` is consumed by the MJCF builder and by the RViz marker publisher. What the operator sees in RViz is geometrically what the robot collides with in MuJoCo; two hand-maintained files would drift, and the drift would look like a perception bug.

Both worlds include a static standing human mannequin, exactly 1.90 m from soles to crown, beside the robot's spawn and clear of its forward travel lane. Its feet rest on the floor; the same primitive geometry appears in MuJoCo and RViz. It provides a size reference rather than a human dynamics model. The [flat-world](#) and [house](#) images show the comparison.

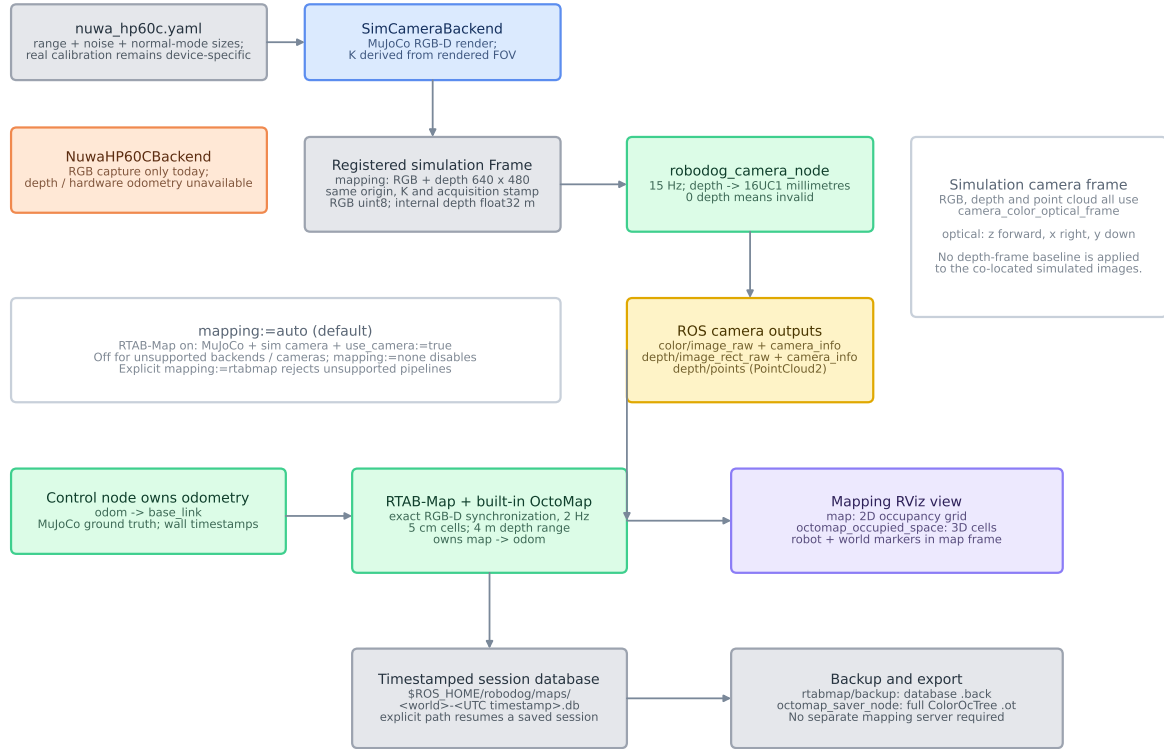
Two worlds ship. `flat` is an open-air proving ground, 24×24 m, with eight independent lanes radiating from the spawn so that each locomotion problem can be attempted on its own: a measured run with metre markers for velocity tuning, a step ladder from 40 to 200 mm, a stair complex (six 80 mm steps up, a landing, six down, and a 130 mm flight beside it), slopes at 5, 10, 15 and 20° , a 196-tile rough-terrain field, stepping stones, a gap course widening from 100 to 280 mm, and a 220 mm balance beam with a pole slalom. Gait work should not be debugged against furniture at the same time, which is why it is separate from the house.

`house` is 12×9 m and contains every indoor locomotion problem the robot must solve, each with enough approach room to be attempted in isolation:

Feature	Dimension	Against
Doorways	0.90 m, 20 mm sill	body width 0.24 m
Narrow gaps	0.45 / 0.43 / 0.60 m	foot span 0.289 m
Stairs	4×80 mm; 3×120 mm	realistic and stretch
Ramp	12°	friction limit at $\mu = 0.9$
Under-table	0.36–0.46 m clearance	stance height 320 mm
Debris field	mixed 40–110 mm blocks	foothold selection
Movable props	4 free bodies	contact recovery

`world_check.py` measures every one of those clearances from the spec, and `test_world.py` fails if an edit turns a designed challenge into a wall or an open field. It also rejects any overlapping geometry, because MuJoCo resolves interpenetration by pushing bodies apart at the first step — silently moving the test environment out from under the experiment.

10 Perception, SLAM and three-dimensional mapping



Simulation uses renderer-derived calibration and exact base odometry. Depth mapping tests do not validate hardware localization or loop closure.

Figure 10: Camera and depth pipeline. Simulated calibration is derived from the renderer; the real camera still requires calibration and depth integration. Mapping uses registered RGB and depth.

The Yahboom NUWA HP60C is mounted on the body’s front face on a bracket at $z = 175$ mm, pitched 0.26 rad down.

Decision — Historical camera mount study

The original camera-mount study predates the revised RS06 geometry and mapping calibration. In that study the front hip assemblies occupied roughly $x = 0.11 \dots 0.26$ m and reached $z = +0.05$ m, leaving only a 42 mm slot between them on the centreline. Rendering the simulated depth image from $z = 20$ mm put **95 %** of the frame on the robot’s own legs at 74 mm; $z = 90$ mm still left 48 %. Clearing the hip corner with the bottom edge of the frustum requires $z > 155$ mm, so the selected mount was raised. At 175 mm the recorded self-view was 0 % and 98.5 % of the depth image was usable. These percentages are historical results, not fresh measurements of the current model.

Decision — The simulated camera loads its own MuJoCo model

The physics instance lives in the control node’s backend. Rather than reach into it, the camera backend loads the same scene read-only and drives it from the published robot state. The camera has a separate node and lifecycle, keeping rendering outside the control callback and camera failures isolated from the control process. Rendering and mapping still compete for workstation resources; the mapping runs below operated slower than real time.

Depth crosses the backend boundary as float32 metres with NaN for invalid, because that representation can express “no reading”; the conversion to the ROS 16UC1-millimetre convention (where 0 means invalid) happens once, at the publisher. Stereo range error grows as z^2 , so the simulated camera applies $\sigma_z = z^2 \sigma_d / (fb)$ — about 4 mm at 1 m and 140 mm at 6 m — and the point cloud is filtered to `max_usable_range_m` rather than the datasheet maximum.

Camera parameters are placeholders

Resolution, field of view, range and baseline in `config/nuwa_hp60c.yaml` are plausible for a device of this class but have *not* been read from the datasheet; each is marked [VERIFY]. The simulated camera instead publishes calibration derived from its rendered field of view and image dimensions; this does not validate the physical device. How the HP60C delivers depth over USB (a second UVC stream, a side-by-side frame, or only through the vendor SDK) is the one thing that cannot be determined without the device, and is marked HW-CHECK in `backends/nuwa_hp60c.py`.

10.1 RGB-D SLAM and 3D occupancy

SLAM estimates a trajectory and corrects the map when places are revisited; OctoMap represents occupied, free and unknown 3D space using those poses. RTAB-Map supplies both the SLAM graph and its built-in OctoMap here, without a separate OctoMap mapping server. The default `mapping:=auto` enables it for `backend:=mujoco`, `camera_backend:=sim` and `use_camera:=true`. It disables mapping for the other sensor pipelines. `mapping:=none` disables processing explicitly; `mapping:=rtabmap` requires a supported configuration and fails early otherwise.

Why RTAB-Map and OctoMap are both present. They solve different parts of the problem. RTAB-Map is the SLAM system: it consumes synchronized RGB, registered depth, camera calibration and odometry, builds a graph of observations, and can correct the graph when the robot revisits a place. OctoMap is the volumetric occupancy representation: it stores probabilistic 3D free/occupied/unknown space in an octree. Running an independent accumulating OctoMap mapper beside graph SLAM would make loop-closure corrections ambiguous, so this stack uses the OctoMap products assembled by RTAB-Map’s map manager.

Frame ownership. Control publishes `odom→base_link` because the walking controller owns base feedback. RTAB-Map is configured with `frame_id=base_link`, `odom_frame_id=odom` and

`map_frame_id=map`; it publishes `map→odom`. This matches the ROS mobile-base convention: `odom` is locally continuous and may drift, while `map` is globally corrected and may jump. The mapper is therefore not a replacement for the state estimator used by control.

Mapping mode renders registered 640×480 RGB and depth from one optical origin. Both images and their calibration use `camera_color_optical_frame` and the acquisition timestamp of the robot state used to render them. Intrinsic follow the rendered camera's vertical field of view and image dimensions. ROS depth remains 16UC1 millimetres. Matching best-effort QoS and exact synchronization preserve this camera contract; repeated stale poses are not republished.

The initial map settings are 5 cm cells, a 4 m depth range and 2 Hz processing. Cell size is resolution, not sensor accuracy. The namespace `/robodog/mapping` contains `map` (2D occupancy), `octomap_occupied_space` (3D points), `octomap_full`, `octomap_binary` and `mapData`. Outputs are generated when subscribed; the map cache remains when viewers disconnect. Mapping RViz uses the `map` frame and shows both 2D and 3D occupancy.

Persistence and export. Each launch creates a preserved, timestamped database for its world:

```
$ROS_HOME/robodog/maps/<world>-<UTC timestamp>.db
# When ROS_HOME is unset:
~/ros/robodog/maps/<world>-<UTC timestamp>.db
```

To deliberately continue or inspect a specific file without deleting any session, pass this launch argument:

```
mapping_database:=/absolute/path/session.db
```

The `/robodog/mapping/rtabmap/backup` service flushes a `.back` copy.

The official `octomap_saver_node` exports a full colored `ColorOcTree` to `.ot` via the node's `octomap_full` service; the database retains the graph and observations, whereas `.ot` is an occupancy snapshot. Exact service remapping and installation commands are in [the mapping runbook](#).

The room-scan service wraps these pieces into one immutable export directory: `cloud_map.pcd` from `/robodog/mapping/cloud_map`, `octomap.ot` from `/robodog/mapping/rtabmap/octomap_full`, `rtabmap.db.back` from `/robodog/mapping/rtabmap/backup`, and a manifest with byte counts and SHA-256 checksums. It refuses stale clouds, relative output roots and existing session names. The export is therefore a scan artifact that can be copied, inspected or archived without mutating the live SLAM database.

Validation and scope. Recorded 0.500 m house and 0.501 m flat walks grew occupied point clouds from 687 to 1,053 and 1,837 to 4,163 points, respectively. Final real-time factors were 0.608 and 0.617. House database reload retained 523 known 2D cells before moving; stored observations increased from 18 to 24 after restart. Full exports contained 7,964 house and 27,567 flat tree nodes, a different count from occupied points. These short integration checks acknowledged stop/stand commands and observed no clamp events during their measurement window; they do not supersede the later failed post-walk stand checks. [The mapping record](#) preserves the measurements and export hashes.

MuJoCo supplies exact odometry, so these runs do not establish real visual odometry accuracy or loop-closure performance. Kinematic mode has no moving base odometry or world spawn placement. The real NUWA backend does not yet provide depth, and real base odometry is not integrated. Hardware mapping needs calibrated registered depth and a validated odometry source with exclusive ownership of `odom→base_link`. This integration does not add autonomous navigation or obstacle avoidance.

11 Operator interfaces and telemetry

A single process holds an `rcipy` node and an `aihttp` server; `rcipy` spins in a background thread while `aihttp` owns the `asyncio` loop. They meet at two dictionaries: the ROS callbacks write the

latest state, the web tasks read it. The browser receives the newest snapshot rather than a backlog. A separate, bounded server-side history computes diagnostics from each received telemetry sample before display throttling.

Decision — A purpose-built protocol rather than rosbridge

rosbridge would re-serialise every field at the full publish rate and expose the whole graph to the browser. This speaks a small versioned JSON protocol, decimated to 20 Hz, and exposes exactly the commands the GUI is allowed to send. Less code, less bandwidth, and a much smaller surface from a page that can drive a robot.

The browser fetches `/api/config` and builds itself from it: adding, removing or reordering a panel is a change to `config/web.yaml` alone. Panels cover robot state, controller state, per-joint telemetry with tracking error and load bars, foot contact and forces, safety and thermal state, simulation state and real-time factor, an event log, camera streams, pose and gait commands, and emergency stop. Expanded robot information describes the configured mass, geometry and actuator/transmission limits; it is not a measurement of the assembled machine.

JOINT RMS, PEAKS & EXPOSURE

12.14 / 20 s covered · 608 samples · 50.0 Hz · mujoco:sim_time

Mujoco applied actuator torque, telemetry sampled. Received telemetry; sampled peaks, not physics-step peaks.

Window resets 0 · rejected samples 0 · sampling gaps 0

Time-weighted RMS and exposure use received telemetry. Absolute peaks can miss short impacts between samples. Exposure compares motor-output torque with the configured continuous limit and the conditional 11 N·m rotating reference. Motor speed at peak is signed. These rolling values do not establish thermal endurance.

JOINT	MOTOR RMS N·M	MOTOR [PEAK] N·M	JOINT RMS N·M	JOINT [PEAK] N·M	MOTOR RPM @ [PEAK]	ABOVE CONT. %	ABOVE 11 N·M %
FL_haa	3.88	10.11	3.88	10.11	2.4	1.34	0.00
FL_hfe	2.05	6.86	2.05	6.86	-2.2	0.00	0.00
FL_kfe	3.56	6.77	6.76	12.86	6.2	0.00	0.00
FR_haa	3.69	9.63	3.69	9.63	0.5	1.18	0.00
FR_hfe	2.13	6.90	2.13	6.90	-2.2	0.00	0.00
FR_kfe	3.49	7.13	6.63	13.55	-8.7	0.00	0.00
RL_haa	4.10	9.75	4.10	9.75	2.2	0.78	0.00
RL_hfe	3.42	7.86	3.42	7.86	-0.1	0.00	0.00
RL_kfe	4.33	8.63	8.23	15.26	-3.6	0.06	0.00
RR_haa	4.05	9.91	4.05	9.91	0.5	0.76	0.00
RR_hfe	3.14	7.54	3.14	7.54	0.7	0.00	0.00
RR_kfe	4.27	7.93	8.11	15.07	-3.8	0.00	0.00

▼ Temperature history, tracking and live controller commands

Temperature is the reported observer/feedback value, not a calibrated prediction. Mechanical power is signed joint torque × speed; negative means mechanical absorption, not measured battery regeneration. Feedforward is not total requested or applied torque; Kp/Kd add feedback torque.

JOINT	WINDOW MAX °C	ERROR RMS RAD	MEAN MECH. W	LIVE KP N·M/RAD	LIVE KD N·M·S/RAD	LIVE FEEDFORWARD N·M	LIVE CMD RAD/S	COVERAGE % / SAMPLES
FL_haa	20.2	0.639	-0.05	90.0	2.00	-3.88	-0.00	12.14 / 608
FL_hfe	20.0	0.625	0.01	90.0	2.00	-0.59	0.36	12.14 / 608
FL_kfe	20.1	0.669	0.83	90.0	2.00	2.31	0.06	12.14 / 608
FR_haa	20.2	0.640	0.01	90.0	2.00	4.38	-0.00	12.14 / 608
FR_hfe	20.1	0.625	0.03	90.0	2.00	1.15	0.42	12.14 / 608
FR_kfe	20.1	0.670	0.97	90.0	2.00	6.80	-0.07	12.14 / 608
RL_haa	20.2	0.656	0.28	90.0	2.00	-6.28	0.00	12.14 / 608
RL_hfe	20.1	0.641	0.47	90.0	2.00	-0.36	0.36	12.14 / 608
RL_kfe	20.2	0.676	2.14	90.0	2.00	3.32	0.01	12.14 / 608
RR_haa	20.2	0.656	0.32	90.0	2.00	9.80	-0.00	12.14 / 608
RR_hfe	20.1	0.638	0.48	90.0	2.00	2.57	0.43	12.14 / 608
RR_kfe	20.2	0.674	1.93	90.0	2.00	8.47	-0.13	12.14 / 608

Figure 11: Live joint diagnostics with rolling torque, thermal and tracking summaries. The displayed values are a mixed-motion interface check, not a steady-gait motor-sizing result.

11.1 Joint load and history conventions

Each of the twelve joints reports live position, velocity, tracking error, signed joint and motor torque, estimated current and temperature. The rolling 20-second summary adds joint/motor RMS and absolute peak torque, motor speed at that peak sample, exposure above the 8 N·m continuous-stall and 11 N·m rotating references, maximum temperature, tracking-error RMS and mean signed mechanical power. Exposure is descriptive, not an allowed overload duration or prediction of failure.

The RS06 internal gearbox is already included in “motor-output torque.” For the external knee belt, $g = 2 \times 0.95 = 1.9$, so $\tau_m = \tau_j/1.9$ and $\dot{q}_m = 2\dot{q}_j$; the hips have no external reduction. A feed-forward command is only one term of impedance control, so its difference from feedback is not a clipping measurement.

The server normally receives telemetry at 50 Hz and updates the browser at 20 Hz. It integrates

a sample over its following valid interval:

$$\tau_{\text{RMS}} = \sqrt{\frac{\sum_i \tau_i^2 \Delta t_i}{\sum_i \Delta t_i}}.$$

MuJoCo statistics use simulation time; other backends use message timestamps. Thus a slow renderer does not inflate exposure. Partial windows show their valid coverage. Gaps over 0.25 s or invalid joint data restart coverage; duplicate and slightly reversed timestamps are rejected. A reversal over one second or clock-source change resets the timeline. The GUI reports these events and leaves missing history unavailable. Browser refresh preserves the server history; server restart resets it. Peaks are telemetry-sampled and can miss contact impacts between messages; the physics-step sizing reports remain the appropriate source for those peaks.

Source provenance. MuJoCo effort is applied physics actuator torque sampled into telemetry; CAN effort is actuator feedback; kinematic effort is synthetic. Current is a torque/ K_t equivalent phase-RMS estimate, not measured phase, battery or i_q current, and excludes high-current saturation. Simulation temperature is an uncalibrated copper-loss model; CAN uses the maximum of reported temperature and its thermal observer. Bus voltage is an assumption and electrical power an estimate. Mechanical power is signed $\tau_j \dot{q}_j$, not battery power. Clamp counts are aggregate command-limiter events, not per-joint actuator clipping. These distinctions are retained in the display and explained in [the diagnostics guide](#).

Operator limits are configuration, not constants in JavaScript: maximum commanded velocity, gait confirmation, and joint jogging — which is disabled by default because it bypasses the gait layer and is the easiest way to put a leg where the body cannot support it. Video is a separate MJPEG endpoint so a slow video consumer cannot stall telemetry, and the page falls back to showing camera metadata if no JPEG encoder is installed.

The simulation-only Greeting action is a queued whole-body motion. One request first selects stand and waits for finite feedback, four contacts, stand height, level/stationary base feedback and settled joints. It then moves the base rear-right, lifts and waves the front-left paw, touches down, and only then recentres. Kinematic regression tests require at least 40 mm support-triangle margin while paw clearance exceeds 60 mm and bound commanded joint speed to 1.25 rad/s. A live 8.70 s MuJoCo run returned to standing with minimum base height 0.294 m, maximum absolute roll 0.067 rad and maximum absolute pitch 0.021 rad. This establishes simulation behavior only; the service still rejects hardware. The reproducible command and full-precision record are in [the greeting validation artifact](#).

11.2 LAN discovery and mobile application

The server publishes a read-only `_robodog._tcp.local` DNS-SD service when `avahi-publish-service` and a running Avahi daemon are available. The service instance carries the friendly name; its TXT records contain a stable UUID, model, API version and the path `/.well-known/robodog`. The JSON descriptor repeats the identity and lists the state, MJPEG and web-remote paths. The UUID is created once below `$ROS_HOME/robodog/device.json` with mode 0600; it is an identifier, not a secret. Discovery can be disabled with `web_discovery:=false`, and manual IP/port entry remains available where multicast is filtered.

The React Native application lives outside the ROS workspace in `apps/robodog_mobile`. Its network boundary validates discovery and telemetry payloads before they enter application state, deduplicates robots by the stable UUID, and keeps motion controls separate from discovery. Phone portrait keeps camera and telemetry above a bottom control area with two side-by-side joysticks. Phone landscape places MOVE and TURN around the live camera and keeps dead-man status plus compact action buttons beside the thumbs. Mode, gait, measured speed, configured supply-voltage assumption, hottest-joint temperature and rolling torque RMS/peak remain visible.

The cockpit’s joysticks with touch dead-man behavior, Stand, Walk, Greeting and emergency stop are active through a hidden, navigation-pinned WebView at `/remote?native_bridge=1`. The robot-served page opens `/ws`, so the browser Origin and Host exactly match the selected robot. The opt-in bridge requires both the query flag and React Native message channel, accepts a strict finite command schema, and returns validated status and telemetry messages. It does not create a standalone native socket or relax the hostname allowlist. Commands retain the server’s one-operator 0.30 s lease and the control node’s independent 0.35 s velocity timeout. Dead-man release, backgrounding and view teardown send zero; disconnect or lease expiry also produces zero. A visible Web Remote remains available for the complete browser interface. Expo Go can use manual address entry; automatic DNS-SD discovery requires the included Swift/Kotlin module in a development build. Setup and operator details are in the [mobile app runbook](#).

Decision — Discovery does not confer control authority

The service advertisement and well-known descriptor are public LAN metadata. They do not add a native command bypass or relax the existing `/ws` Origin, Host or private-network checks. The hidden WebView bridge deliberately uses those existing checks. The shipped HTTP/MJPEG/web-control path has no login or TLS and is therefore restricted to an isolated trusted LAN. A production control layer must add TLS, pinned robot identity, a time-limited pairing code, revocable per-device credentials, strict message schemas, rate limits and a lease covering every motion-changing action.

The present camera path is not a 60-fps path: ROS publishes the simulated camera at 15 Hz and the Python MJPEG encoder is configured for at most 10 Hz at 640 pixels wide. A later low-latency feed belongs behind a separate media transport using Jetson hardware H.264 and WebRTC with native decoding. Camera sensor output, encoder throughput, decoded frame rate, latency, packet loss, CPU/GPU load and temperature must be measured before a 60-fps profile is advertised. Command telemetry remains on a separate transport so video congestion cannot delay dead-man messages.

12 Experimental methodology and results

Current configuration. The active source is `cad/urdf/urdf/urdf.urdf`, with twelve RS06 actuators and a working mass of exactly 28.000 kg in the loaded test model. This combines the 19.719271 kg base configuration with 8.280729 kg of centered, removable simulation ballast. Generated geometry, inertias and actuator macros below describe the active model. The external knee belt is 2:1 with an assumed 95 % efficiency: the 8 N·m continuous-stall reference becomes 15.2 N·m at the knee. The 11 N·m rotating rating becomes 20.9 N·m, and 36 N·m peak becomes 68.4 N·m. These are transmission calculations, not belt or shaft strength validation.

Historical measurements are superseded

Sections explicitly labeled historical, and legacy stance-height and terrain examples, describe the earlier 10 kg RS02 model. They are retained as engineering history, not acceptance evidence for the RS06 configuration. Current architecture diagrams reflect the RS06 configuration. Use the current `tools/torque_report.py` output for new per-joint measurements; generated macros update configuration facts but do not rerun empirical experiments.

Cooling, overload and power assumptions. The manufacturer’s September 17 specification rates RS06 at 11 N·m at 100 rpm with a 200 × 200 mm aluminum heat sink. Its separate stall table specifies 8 N·m continuous; the controller uses that holding value as its continuous budget. At 36 N·m the maximum is 1 second stalled or 4 seconds rotating. These are published test points, not permission to repeat peak pulses indefinitely. Installed cooling and recovery remain unverified.

The latest source publishes line resistance and equivalent output inertia; thermal impedance, friction and belt losses still require identification. Simulation temperatures remain estimates. The battery assumption is two 6S packs in series: 44.4 V nominal and 50.4 V fully charged. Pack wiring and discharge behavior must be verified separately from mass. The central electronics plate does not establish equivalent heat removal for twelve actuators.

Hardware status. No hardware has been driven. Direction, zero reference, firmware scaling, wiring and cooling require physical validation. The real IMU/base-state input is not integrated. Missing feedback remains explicitly unavailable, and travel commands are refused until live base feedback is connected. Hardware startup defaults to disabled; RS06 normal enable/motion requires joints marked calibrated after physical commissioning. MuJoCo supplies exact base feedback, so successful simulated motion does not validate the physical estimator. Turning tests below expose tracking error; stairs, jumps, pace and bound still need separate validation.

Sources. Configuration facts come from `robot_parameters.yaml` and the file selected by its `actuator_config`, through `tools/make_doc_facts.py`. Performance and electrical values follow the manufacturer’s September 17, 2026 RS-series specification; the July RS06 manual supplies protocol details. [The RS06 datasheet audit](#) records source links, page references, source hashes and differences between these documents. The research pass behind this revision is captured in [docs/research/robodog_scientific_foundations.md](#); external claims below cite primary project pages, specifications, source code or peer-reviewed robotics papers rather than secondary tutorials.

Current gait settings and report. Walk uses 1.5 Hz, duty factor 0.78, 30 mm swing clearance and 320 mm stance height. Trot uses 2.2 Hz, 40 mm clearance and 330 mm stance. Stance impedance remains 90/2 and swing impedance 60/1.5 in joint-space stiffness/damping units. Current loaded-model measurements and their limits are in [docs/rs06_28kg_1ms_100s.md](#); its JSON, CSV and MuJoCo XML retain all twelve joint results and the evaluated model. The historical 19.719271 kg sweep remains in [docs/rs06_speed_study.md](#). The GUI proving ground’s barrier at 9 m makes it unsuitable for these longer speed runs. [The locomotion validation report](#) compares speeds, turning and the official Go2 model.

12.1 Contributions and research basis

RoboDog is not a finished product; it is a research and bring-up platform whose most important deliverable is a traceable model-to-controller chain. The architecture contributes five concrete pieces:

1. a generated mechanical description in which CAD, URDF, MuJoCo, inverse kinematics and documentation share one parameter source;
2. a backend boundary that lets the same controller drive ideal kinematics, MuJoCo dynamics or RS06 CAN hardware;
3. a foot-trajectory gait controller with force-distribution feed-forward, motor-side load accounting and per-cycle safety limiting;
4. an RGB-D mapping pipeline that uses RTAB-Map for graph SLAM and its graph-corrected OctoMap products for room scanning;
5. a measurement method that reports applied physics torque, motor-side RMS, peaks, overload exposure, speed at spikes and clipping separately.

Topic	Primary source used	How it constrains this system
ROS frames	REP-103 and REP-105 [9, 10]	SI units, body and camera optical axes, and ownership of <code>map</code> , <code>odom</code> and <code>base_link</code> .
Actuation in simulation	MuJoCo computation documentation [11]	Applied torque is read from <code>qfrc_actuator</code> , after actuator force generation and transmission mapping into generalized coordinates.
RGB-D SLAM	RTAB-Map ROS 2 README and source [14, 15]	RTAB-Map owns graph SLAM, database persistence and map products; the launch here does not run a separate odometry broadcaster or separate OctoMap mapper.
3D occupancy	OctoMap project and ROS 2 saver source [16, 17]	<code>.ot</code> export is an occupancy snapshot from a <code>GetOctomap</code> service, not the SLAM database.
Quadruped locomotion	Raibert-style foot placement and MIT Cheetah literature [18, 19]	Velocity regulation is treated as foot-placement and contact-force control rather than joint-position playback alone.
Benchmarking	Official <code>unitree_mujoco</code> repository [20]	Go2 comparison uses the official MJCF under the same measurement method, not a manufacturer factory controller claim.

Experimental method. A feasible operating point must complete the run, stay upright, track commanded body velocity, keep worst motor RMS below the configured continuous-stall reference and avoid meaningful command intervention by the safety limiter. RMS is computed after warmup as

$$\tau_{\text{RMS}} = \sqrt{\frac{\int_{t_0}^T \tau_m(t)^2 dt}{T - t_0}},$$

where τ_m is actuator-output torque before any external knee belt. Peak, time above 8/11/36 N·m, speed at peak and clipping durations include startup because startup is part of the commanded maneuver. That separation is intentional: RMS estimates heating tendency, while peak and exposure describe shock, overload and limiter stress.

12.2 Current RS06 simulation measurements

The current 28.000 kg loaded-model cases ran for 100 simulated seconds at a 1.00 m/s command, with a five-second RMS warmup. The walk case falls: actual speed is -0.01096 m/s, maximum tilt is 33.17 degrees, minimum body height is 0.14857 m, and worst motor RMS/peak is 7.681/32.159 N·m. The heading-hold trot stays upright but reaches only 0.922506 m/s; maximum tilt is 8.6559 degrees, minimum height is 0.32165 m, and worst RMS/peak is 6.749/34.897 N·m. Its largest spike is rear-left hip pitch at 0.681 s, after 0.4776 m travel, at world position [0.467, 0.054, 0.323] m.

Safety changes commands during 99.82 % of walk cycles and 87.965 % of trot cycles. Walk reports position, velocity and torque fault bits; trot reports velocity and torque bits. Neither case passes the tracking and motor-margin criteria. No peak-torque or MuJoCo actuator clipping occurs, but extensive safety and modeled speed-envelope limiting remains. Temperatures are

uncalibrated estimates. Against the historical 19.719271 kg trot, actual speed falls from 0.984 to 0.923 m/s, RMS rises from 5.360 to 6.749 N·m, peak rises from 28.638 to 34.897 N·m and safety-limited cycles rise from 30.6 % to 88.0 %.

Historical 19.719271 kg speed sweep. Each case ran for 20 seconds with the configured gains, motor envelope and safety monitor active. Commands step from a standing keyframe with 2 mm ground clearance. The controller runs at 400 Hz, physics at 2 kHz, and command delay is two physics steps (1 ms). Applied actuator torque is read from MuJoCo `qfrc_actuator` every physics step, after limits and transmission gears; it is neither the requested PD effort nor the total structural/contact load. RMS excludes the first second; peaks and overload durations include startup. For knees, inferred motor-output torque is joint torque divided by 2×0.95 ; hips are direct drive. These measurements superseded earlier RS06 results collected before the delay correction and physics-rate sampling; they now serve as base-configuration comparison data for the 28 kg loaded model.

Gait	Command (m/s)	Actual (m/s)	Motor RMS (N·m)	Motor peak (N·m)	Clamped (% cycles)	Stable/tracks
Stand	0.00	≈0.000	3.28	4.05	0.0	yes/yes
Walk	0.15	0.171	4.94	10.77	0.0	yes/yes
Walk	0.40	0.420	5.36	23.59	0.0	yes/yes
Walk	0.50	0.521	5.91	29.30	60.6	yes/yes
Trot	0.40	0.414	4.73	11.78	0.0	yes/yes
Trot	0.50	0.515	4.79	13.34	0.0	yes/yes
Trot	0.75	0.756	4.99	21.19	0.0	yes/yes
Trot	1.00	0.979	5.38	28.15	26.8	yes/yes
Trot	1.50	1.477	6.49	30.37	99.4	yes/yes
Trot	2.00	0.052	7.21	35.12	99.9	no/no

RMS and peak are the worst individual motor values and may belong to different joints. The clamped column counts safety-limited control cycles; separate motor-envelope reductions and fault flags are retained in the full report. Walk at 0.4 m/s tracks but raises a velocity-limit flag; at 0.5 m/s it frequently activates safety limiting. Walk commands from 0.75 through 2.0 m/s fail stability/tracking. Trot remains stable through the tested 1.5 m/s command, but limiting starts at 1.0 m/s and affects almost every cycle at 1.5 m/s. Trot at 2.0 m/s fails. Failed-run loads include the fallen portion and are not successful operating points. Accepting a 2 m/s GUI command therefore does not establish that the robot can execute it.

At 0.4 and 0.5 m/s trot, worst motor RMS is 59 % and 60 % of the 8 N·m continuous-stall reference, respectively. The separate 11 N·m rotating rating requires the specified heatsink. These short runs do not validate installed cooling, belt/pulley strength, mounting fit, battery sag, regenerative power or physical sensor integration. Estimated thermal values remain uncalibrated.

Turning. With a requested yaw rate of 0.30 rad/s, the 0.4 and 0.5 m/s trot cases achieve only 0.127 and 0.144 rad/s, respectively. They stay upright and track forward speed, but fail the requested turn rate. Feedback and forward-speed measurements use the body’s yaw frame.

Go2 comparison and system smoke test. The pinned official Go2 MJCF benchmark uses the same plane, effective contacts, solver, timestep, 20-second duration, 1-second RMS warmup, startup clearance, delay and shared trot generator/gains. Go2’s 15.206 kg model retains its native inertias and torque caps; Robodog retains its RS06 envelope and safety policy. This is a shared-controller model comparison, not Unitree factory-controller performance. Both floating bases move only through joint actuation and ground contact. At 0.4/0.5 m/s, Go2 achieves 0.393/0.492 m/s; its worst knee joint RMS is 7.95/8.02 N·m versus Robodog’s 8.99/9.10 N·m. Compare joint-output

effort before separately accounting for Robodog’s knee belt. [The official viewer/SDK runner test](#) advances 781 steps on loopback DDS, then exits with a segmentation fault during shutdown; it is not a clean runtime pass. The headless benchmark completes normally. The shutdown failure remains unresolved.

Latest integration status. The subsequent GUI and default-mapping checks passed all 364 regression tests and exercised the diagnostics in a live browser. A separate requirement for a fault-free stand after walking **did not pass**: both house and flat runs reported `TORQUE_LIMIT` after the transition, with increasing aggregate limiter counts. The house capture identifies the rear-right hip-roll joint at its configured continuous budget. This unresolved controller/load issue is a different scenario from the successful sustained-gait benchmarks above; those benchmarks do not establish comfortable margins for every transition. [The GUI validation record](#) retains both the passing interface checks and the failed transition assertion.

13 Hardware migration and commissioning

13.1 What changes

Component	Simulation	Hardware
Joints	MujocoBackend	RobStride06Backend
Camera	SimCameraBackend	NuwaHP60CBackend
Base state	ground truth from the simulator	StateEstimator
Temperature	thermal model	reported, with the model as observer
Launch	<code>backend:=mujoco</code>	<code>backend:=robstride06_can</code>

Core command and telemetry interfaces remain shared. Hardware still requires physical commissioning, base-state integration and camera calibration. The optional mapping node stays disabled until registered depth and odometry are available; simulation gains and limits do not certify hardware operation.

13.2 Bring-up sequence

1. **Verify the CAN protocol.** Power one motor alone; confirm `GET_DEVICE_ID`, then `ENABLE` and the arrival of feedback frames, then a 0.1 N·m command with $k_p = k_d = 0$ and check the direction of motion. A firmware revision with a different identifier layout changes exactly one file.
2. **Calibrate each joint.** `ros2 run robodog_hardware calibrate_joint` determines `direction` and `offset_rad` against the convention in Section 3.3 and prints a YAML fragment for review. Leave `enable_on_start: false` until signed off.
3. **Validate the remaining assumptions.** Check assembly dynamics against the published output-equivalent inertia; identify friction, damping and cooling through controlled measurements within manufacturer limits. The 11 N·m rotating rating is not a continuous stall allowance. Until then the margins in Section 7 carry the uncertainty.
4. **Bring up on a stand**, legs free: joint test sweeps, then named poses, then stance with the body supported.
5. **On the ground**, walk gait first — it is statically stable and keeps three feet down.

13.3 Known gaps

Item	Status	Resolution
CAN frame layout	from the RS06 manual	verify against delivered firmware
Joint direction / offset	unknown	<code>calibrate_joint</code> , per joint
Output equivalent inertia	published 0.012 kg·m ²	validate assembly dynamics
Damping, Coulomb friction	estimate	swing-down test
Thermal R_{th} , C_{th}	estimate	controlled heating/cooling tests
Temperature thresholds	estimate	from the above, plus vendor guidance
Camera intrinsics, range	placeholder	datasheet, then <code>camera_calibration</code>
HP60C depth transport	unknown	determine with the device
Joint limits	design values	self-collision check on real geometry
Thigh/base clearance	contact excluded	confirm in CAD; see §6.6
Battery model	nominal voltage only	add a cell model and state of charge

13.4 Planned next steps

The `ros2_control` interface contract is already declared in `urdf/ros2_control.xacro` but not implemented: the runtime is Python.

Decision — Python control loop now, C++ `SystemInterface` later

At 400 Hz the measured loop period is 2.49 ms against a nominal 2.50 ms, with worst-case jitter under 1 ms — adequate, and it buys a stack that is fully testable and quick to change during architecture work. The declared `ros2_control` block is the formal statement of the interfaces a C++ `SystemInterface` must provide, and is the migration path once jitter or CPU headroom demands it. Note that the command interface set there is the RS06’s native impedance command, not a single position interface, for the reason in Section 5.

14 Reproducibility and verification

The latest recorded regression run on 28 September 2026 passed all 364 tests. The statistics module has 100 % statement coverage, including irregular timing, belt conversion, partial windows, invalid samples, resets and bounded history; browser tests keep unavailable values distinct from zero. Camera tests reconstruct the rendered floor using published calibration. Live Chrome checks populated a full 20-second history at approximately 50 Hz without browser errors or telemetry gaps and verified default mapping in both worlds. The separate fault-free post-walk stand assertion failed with `TORQUE_LIMIT`; passing regression tests do not resolve that load issue. All launched integration-test nodes exited cleanly.

Run the suite again for a changed checkout. The key checks cover URDF/MJCF consistency, joint-to-motor conversion, CAN scaling, bounded commands, free hip motion, continuous gait targets and actual simulated displacement. Tests must compare knee motor load after the belt conversion, not directly against joint torque. Record the command, test count and skips with each feasibility report.

The most valuable are the cross-checks: the MJCF’s per-link masses, joint origins, axes and ranges are asserted against the URDF, so the two descriptions cannot drift. Seven tests exist specifically because a bug got past a weaker one: the gravity-torque sign, the idle-command rate-limiter seed, the camera self-occlusion fraction, the Euler-convention mismatch that left the rendered legs beside the body while every physics test passed, the swing arc’s discontinuity at lift-off, the self-collision that welded the front hips, and the gait’s thermal load.

The last three describe historical RS02 regression work. That thermal test spent most of the project as an expected failure at 173 % of continuous, and it did not turn green by being tuned

— it turned green when the two bugs above it were fixed, and that historical run read 90 %. An expected-failure test is a good way to hold a known gap where a counter will find it, but it says nothing about *why* the gap exists: the reason sat two layers below where the test pointed, in the simulation model.

14.1 Simulation quick start

With ROS 2 Humble installed and from the repository root:

```
source /opt/ros/humble/setup.bash
python3 -m pip install --user mujoco aiohttp
sudo apt-get install ros-humble-rtabmap-slam \
  ros-humble-rtabmap-util ros-humble-rtabmap-sync \
  ros-humble-octomap-server
cd ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 run robodog_sim generate_models
ros2 launch robodog_bringup robot.launch.py \
  backend:=mujoco world:=house
```

Open <http://localhost:8080> on the robot, or <http://<robot-ip>:8080/remote> from another device on the same trusted LAN. Use `web_host:=127.0.0.1` for local-only access. Additional launch arguments:

- `world:=flat`: use the proving ground.
- `mujoco_viewer:=true`: open the physics viewer.
- `mapping:=none`: measure control without mapping load.

The OctoMap server package supplies the export utility; no independent OctoMap mapping server starts. `colcon build` does not install missing Python packages: `python3 -c 'import aiohttp, mujoco'` must succeed in the sourced terminal. See [the mapping runbook](#) for database backup and export, and [the diagnostics guide](#) for measurement interpretation.

14.2 Analysis tools

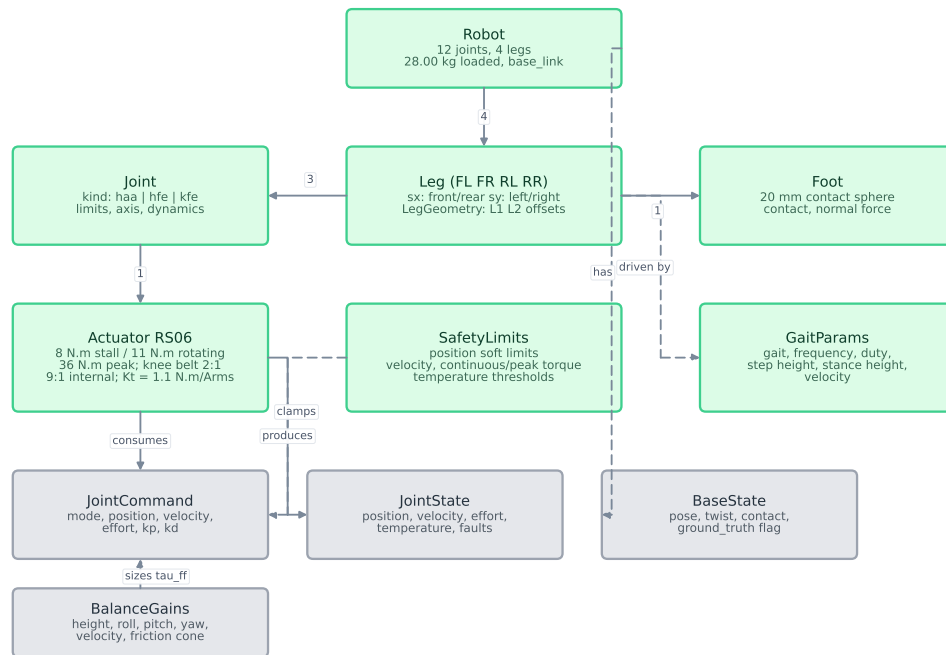
Repository tools produce the simulation measurements and generated configuration facts quoted here. Manufacturer ratings retain their source audit; estimates and hardware gaps remain explicitly identified.

Tool	Answers
<code>cad_to_model.py</code>	CAD → canonical model; prints the FK residual
<code>prepare_meshes.py</code>	active CAD mesh conversion and decimation
<code>torque_report.py</code>	peak and RMS torque per gait, and the winding temperature
<code>capability_report.py</code>	static point-load and geometric reach screening
<code>design_study.py</code>	torque against leg length and stance height
<code>optimise_stance.py</code>	the stance-pose search behind the named poses
<code>make_diagrams.py</code>	the ten figures, as Draw.io and PDF
<code>make_torque_figure.py</code>	why torque follows leg bend, not leg length
<code>make_doc_facts.py</code>	configuration macros; does not rerun experiments
<code>render_previews.py</code>	the MuJoCo preview renders

14.3 Capability envelope

The current `capability_report.py` provides a static point-load screen with the active mass and per-joint belt conversion. It assumes equal supporting foot loads and omits limb gravity, acceleration and asymmetric contacts. It therefore does not certify payload, stair climbing, slope traction,

jumps or landings. Reach reserve is reported as geometry, not as demonstrated step height. Use the full gait report and dedicated terrain experiments for dynamic claims. Thermal feasibility also requires identified cooling on the assembled robot; the estimated copper-loss model is insufficient by itself.



Value types crossing the hardware boundary and the entities that own them.

Figure 12: Domain model: the value types that cross the hardware boundary and the entities that own them.

15 References

References

- [1] M. H. Raibert, *Legged Robots That Balance*, MIT Press, 1986. <https://mitpress.mit.edu/9780262681193/legged-robots-that-balance/>
- [2] M. H. Raibert, “Dynamically Stable Legged Locomotion: Second Report to DARPA,” CMU Robotics Institute, 1983. <https://publications.ri.cmu.edu/dynamically-stable-legged-locomotion-second-report-to-darpa-october-11981-december-311982>
- [3] O. Khatib, “The Operational Space Formulation in Robot Manipulator Control,” 1985. https://cs.stanford.edu/group/manips/publications/pdfs/Khatib_1985_ISIR.pdf
- [4] N. Hogan, “Impedance Control: An Approach to Manipulation: Part I—Theory,” 1985. <https://doi.org/10.1115/1.3140702>
- [5] Open Robotics, “ROS 2 nodes documentation source.” https://github.com/ros2/ros2_documentation/blob/rolling/source/ROS-Framework/About-Nodes.rst
- [6] Open Robotics, “ROS 2 topics documentation source.” https://github.com/ros2/ros2_documentation/blob/rolling/source/ROS-Framework/interfaces/About-Topics.rst
- [7] Open Robotics, “ROS 2 Quality of Service documentation source.” https://github.com/ros2/ros2_documentation/blob/rolling/source/ROS-Framework/interfaces/topics/About-Quality-of-Service-Settings.rst

- [8] ROS project, “robot_state_publisher README.” https://github.com/ros/robot_state_publisher
- [9] T. Foote and M. Purvis, “REP-103: Standard Units of Measure and Coordinate Conventions,” ROS Enhancement Proposals. <https://github.com/ros-infrastructure/rep/blob/master/rep-0103.rst>
- [10] W. Meeussen, “REP-105: Coordinate Frames for Mobile Platforms,” ROS Enhancement Proposals. <https://github.com/ros-infrastructure/rep/blob/master/rep-0105.rst>
- [11] Google DeepMind, “MuJoCo computation documentation: actuation model and generalized forces.” <https://mujoco.readthedocs.io/en/stable/computation/>
- [12] ROS 2 common interfaces, “sensor_msgs/msg/PointCloud2 message definition.” https://github.com/ros2/common_interfaces/blob/rolling/sensor_msgs/msg/PointCloud2.msg
- [13] M. Labbe and F. Michaud, “RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation.” <https://arxiv.org/abs/2403.06341>
- [14] IntRoLab, “rtabmap_ros: ROS 2 wrapper for RTAB-Map.” https://github.com/introlab/rtabmap_ros/blob/ros2/README.md
- [15] IntRoLab, “rtabmap_slam CoreWrapper source.” https://github.com/introlab/rtabmap_ros/blob/ros2/rtabmap_slam/src/CoreWrapper.cpp
- [16] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss and W. Burgard, “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, 2013. Project page: <https://octomap.github.io/>
- [17] OctoMap project, “ROS 2 octomap_saver source.” https://github.com/OctoMap/octomap_mapping/blob/ros2/octomap_server/src/octomap_saver.cpp
- [18] J. Di Carlo, B. Katz, G. Bledt and S. Kim, “MIT Cheetah 3: Design and Control of a Robust, Dynamic Quadruped Robot,” MIT records. <https://dspace.mit.edu/entities/publication/b77b4b14-6d3d-43c0-899e-e85ddbba2a8f>
- [19] J. Kim, J. Kim, S. Kim and S. Kim, “Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and Model Predictive Control,” 2019. <https://arxiv.org/abs/1909.06586>
- [20] Unitree Robotics, “unitree_mujoco official repository.” https://github.com/unitreerobotics/unitree_mujoco
- [21] RoboDog project, “RS06 manufacturer datasheet audit,” `docs/rs06_datasheet_audit.md`.
- [22] RoboDog project, “Scientific foundations primary-source notes,” `docs/research/robodog_scientific_foundations.md`.

Generated from `robot_parameters.yaml` (CAD b95d92fd9159f4f7) and `robstride06.yaml`; research notes `docs/research/robodog_scientific_foundations.md`.
Diagrams: `docs/diagrams/*.drawio`, rendered by `tools/make_diagrams.py`.